International Journal of Information Security (2026) 25:88
https://doi.org/10.1007/s10207-026-01237-x

REGULAR CONTRIBUTION

EdgeFence: Federated Temporal Graph Neural Networks for Lightweight, Adversarial Malware Detection in Distributed Edge Networks

Osei Isaac[1] · Benjamin Appiah[2] · Daniel Commey[3] · Kwabena Owusu-Agyemang[1] · Michael Asante[1] · Benjamin Hayfrom Acquah[1]

Received: 20 August 2025 / Accepted: 18 February 2026 / Published online: 5 May 2026
© The Author(s) 2026

Abstract

The rise of malware in highly interconnected and resource-limited distributed edge networks poses a considerable challenge for traditional security measures. Effective malware detection in these environments requires real-time analysis capabilities, minimal computational overhead on edge devices, strong resilience against adversarial evasion techniques, and the preservation of data privacy across distributed nodes. This paper presents EdgeFence, an innovative framework aimed at lightweight adversarial malware detection within distributed edge networks, utilising Federated Temporal Graph Neural Networks (FTGNNs). EdgeFence represents the dynamic behaviour of processes and system interactions at individual edge nodes through the use of temporal graphs. In contrast to centralised methods, it utilises a federated learning framework, enabling edge devices to work together in training a global detection model by exchanging model updates instead of raw data, which helps maintain data privacy and minimises communication overhead. A significant contribution is the incorporation of Temporal Graph Neural Networks refined for efficiency, adept at capturing sequential dependencies and structural anomalies in dynamic graph data streams produced at the edge. Additionally, EdgeFence integrates adversarial training methods into the federated learning framework to improve the model's resilience against advanced malware intended to bypass GNN-based detection. Our evaluation shows that EdgeFence attains high accuracy and low false positive rates in detecting various malware families in real-time on resource-limited edge devices, while also demonstrating considerable resilience to adversarial attacks. EdgeFence offers a practical and scalable solution for securing large-scale distributed edge computing infrastructures against evolving cyber threats, thanks to its lightweight architecture and federated learning approach.

Keywords Federated Learning · Edge Computing · Malware Detection · Graph Neural Networks (GNNs) · Adversarial Robustness

✉ Benjamin Appiah
kubipph7@gmail.com

Osei Isaac
oseiiassac@yahoo.com

Daniel Commey
dcommey@tamu.edu

Kwabena Owusu-Agyemang
koagymang@knust.edu.gh

Michael Asante
masante@knust.edu.gh

Benjamin Hayfrom Acquah
bhaquah@knust.edu.gh

[1] Dept. Computer Science, Kwame Nkrumah University of Sci. Tech, Kumasi, Ghana

[2] Dept. Computer Science, Ho Technical University, Ho, Ghana

[3] Dept. Multidisciplinary Engineering, Texas A&M University, Texas, USA

Mathematical Symbols and Notations

i	Index for a specific edge device or client
N	Total number of edge devices/clients in the system
ΔT	Fixed time interval for constructing graph snapshots
t	Time step
$G_{i,t}$	Local graph snapshot for device i at time step t
$V_{i,t}$	Set of nodes (entities) in graph $G_{i,t}$
$E_{i,t}$	Set of edges (interactions) in graph $G_{i,t}$

$X_{i,t}$	Node features for graph $G_{i,t}$
$\|V_{i,t}\|$	Number of nodes in graph $G_{i,t}$
d_v	Dimension of node features
w	Temporal window size (number of recent graph snapshots in a sequence)
$\mathcal{G}_{i,t,w}$	Sequence of the w most recent graph snapshots for device i at time t
f_θ	The lightweight Temporal Graph Neural Network (TGNN) model
θ	Parameters of the TGNN model f
L	Number of GNN layers within the TGNN architecture
t'	A specific time step within the temporal window $\mathcal{G}_{i,t,w}$
$H_{i,t'}^{(l)}$	Node embeddings at layer l for the graph snapshot $G_{i,t'}$
$\text{GNNLayer}^{(l)}(\cdot,\cdot)$	Function representing the operations of GNN layer l
$A_{i,t'}$	Adjacency matrix for graph snapshot $G_{i,t'}$
AGGREGATE	Function within a GNN layer for aggregating neighbor information
COMBINE	Function within a GNN layer for combining aggregated information
$\sigma(\cdot)$	Non-linear activation function
$H_{i,t'}^{(L)}$	Final node embeddings after L GNN layers for graph snapshot $G_{i,t'}$
$h_{G_{i,t'}}$	Snapshot-level representation obtained from $G_{i,t'}$
READOUT$(\cdot)$	Global readout function for obtaining a snapshot representation
$\mathbf{h}_{i,t}^{temp}$	Final temporal embedding for device i at time t
$s_{i,t}$	Maliciousness score output by the TGNN for device i at time t
$f_{\theta_{global}}$	The global TGNN model with parameters θ_{global}
θ_{global}	Parameters of the global TGNN model
$\Delta t_{inference}$	Time taken for real-time inference on one temporal sequence
τ_{detect}	Detection threshold applied to the maliciousness score $s_{i,t}$
r	Index for federated learning rounds
$\theta^{(r)}$	Global model parameters at the beginning of federated learning round r
$\mathcal{S}^{(r)}$	Subset of clients selected to participate in federated learning round r
θ_i	Local model parameters for client i
$\mathcal{D}_i$	Private local dataset of temporal graph sequences for client i
$\mathcal{G}_{i,t,w}^{(j)}$	The j-th temporal graph sequence sample in the local dataset $\mathcal{D}_i$
$y_i^{(j)}$	Ground truth label for the j-th temporal graph sequence in $\mathcal{D}_i$
$\mathcal{G}_T$	Generic notation for a temporal graph sequence sample
y	Generic notation for the ground truth label of a temporal graph sequence
$\mathcal{L}_i(\theta)$	Empirical risk (loss function) for client i with parameters θ
$\ell(\cdot,\cdot)$	The loss function (e.g., binary cross-entropy)
E_{local}	Number of local training epochs performed by clients per FL round
$\theta_i^{(r+1)}$	Updated local model parameters for client i after local training in round r
$\Delta\theta_i^{(r)}$	Local model update from client i in round r $(\theta_i^{(r+1)} - \theta^{(r)})$
$\nabla\mathcal{L}_i(\theta^{(r)})$	Local gradients of the loss for client i at parameters $\theta^{(r)}$
$\theta^{(r+1)}$	Global model parameters after aggregation in federated learning round r
$G'_{i,t,w}$	Adversarial version of the temporal graph sequence $\mathcal{G}_{i,t,w}$
δ	Adversarial perturbation applied to a temporal graph sequence
$\delta_{t'}$	Perturbation applied to a single graph snapshot $G_{i,t'}$ within a sequence
$G'_{i,t'}$	Perturbed graph snapshot at time t'
Δ	Predefined threat model (set of allowed perturbations)
$\delta^*(\mathcal{G}_T, \theta_i)$	Optimal adversarial perturbation for $\mathcal{G}_T$ under threat model Δ given model θ_i
λ	Weighting factor for the adversarial loss term
$\mathcal{L}_i^{adv}(\theta_i)$	Composite local loss function including adversarial training

1 Introduction

The latest edge computing trends have been changed with managing processing to distributed devices that are resource-constrained and located near the data sources. These devices can be used for applications that need low latencies such as smart factories and autonomous car driving; at the same time, they bring a lot of new security challenges because the whole attack surface grows very fast. Those edge devices are interesting targets for cyber attacks but have very little processing power for typical centralized security solutions such as signature-based detection or cloud-dependent AI. Furthermore, these systems face detection problems with new malwares, extensive sending and latency issues for data transmission, and major privacy concerns due to central

aggregation [1–17]. As a result, there exists a dire requirement for independent, efficient, and self-sustaining security in its capacity to operate directly on the edge.

Modeling system activity as graphs provides program behavior in a powerful relational view that captures well the complex interactions typical of malware. Graph Neural Networks (GNNs) are well suited to learning from such structures and have demonstrated efficacy in security tasks [18]. However, malware progresses in time, thus requiring an analysis of dynamic temporal graph sequences to infer which attacks are stealthy and which meritorious. Standard GNNs and their dynamic versions usually have heavy computational load, which presents a hindrance to deployment onto low-power edge devices. Hence, there is a need for lightweight and efficient models for real-time and on-device analysis of evolving behavioral graphs.

An additional need arises for collaborative yet privacy-preserving defense strategies addressing the distributed nature of edge networks. Federated Learning (FL) provides an encouraging framework for devices to train a global AI model collaboratively by sharing only model updates and not raw private data [19–21]. At the same time, the increasing threat of adversarial attacks dealing with the target evasion of machine learning models demands the development of secure AI that is robust and resilient. A feasible solution must therefore implement a dynamic and efficient graph analysis for threat detection on the devices with federated cooperation and adversarial robust to provide dependability to critical edge applications.

Addressing these multifaceted challenges,, we propose **EdgeFence**, a novel federated learning framework designed for lightweight, adversarial malware detection tailored for distributed edge networks. The combination of a privacy-preserving federated learning approach with a temporal graph neural network (TGNN) enables real-time dynamic analysis of program and system call behaviors on edge devices through EdgeFence. Utilizing a lightweight TGNN architecture with optimized graph processing, EdgeFence minimizes the computation and memory footprint on the resource-constrained edge nodes. More importantly, the framework directly incorporates adversarial training techniques into the federated learning loop, thereby greatly augmenting the global model's robustness to evasion strategies, including those that obfuscate with Generative Adversarial Networks (GANs). Our experimental evaluation on real-world edge malware datasets demonstrates that EdgeFence achieves competitive detection accuracy and significantly lower operational overhead compared to centralized GNN-based methods and existing federated learning baselines, while exhibiting strong robustness against adversarial samples. Our work makes the following contributions:

1. We propose a novel federated learning framework that integrates Temporal Graph Neural Networks for privacy-preserving, distributed malware detection in simulated edge environments.
2. We develop a lightweight Temporal GNN architecture designed to support real-time analysis of dynamic system behavior in environments representative of resource-constrained edge scenario.
3. We further integrate adversarial training into the federated learning process to enhance detection robustness against adversarial malware in distributed simulated edge scenario.

2 Research Background

2.1 Problem Statement

The world of computing is increasingly moving away from big, central data centers and out to the "edge" of the network. This means we now have a massive number of smaller devices scattered everywhere. Each of these gadgets has its own strict limits on processing power (C_{limit}), memory (M_{limit}), and often, how much battery it can use (P_{limit}). While this big change helps run applications that need lightning-fast responses right at the source, it also creates a much larger and more diverse playground for cyberattacks. The harmful software being created for these edge devices is getting much more sophisticated. It uses clever tricks to constantly change its appearance, making it hard to recognize, and can even coordinate its attacks across many different hacked devices at once. To properly defend this new, spread-out landscape, we need a new kind of security system. It must be able to analyze threats as they happen, right on the device, without invading user privacy or overwhelming the device's limited resources.

The classic security model, where all the information from edge devices is sent back to a big, powerful central server for a check-up, just isn't cut out for this new world. The constant firehose of data coming from a huge number of devices is simply too much. It would completely jam the available internet connection (B_{uplink}) and quickly run out of storage space on the main server. Beyond the logistical nightmare, pulling in all the raw operational data from so many different devices creates massive privacy and legal headaches, especially in fields with strict rules. What's more, there's always a time lag (τ_{comm}) as information travels from the device to the server and back. This delay makes a centralized system useless for stopping threats that unfold in the blink of an eye. For security to be effective, you have to spot the problem (Δt_{detect}) and react faster than the critical window you have to prevent damage (δ_{response}), and the old way is just too slow.

Although running security checks right on each edge device gets around the speed and data bottleneck problems, we hit another major wall: trying to cram a powerful, modern AI model onto a piece of hardware that's just not built for it. The latest and greatest detection methods, especially those using Deep Learning, are incredibly hungry for processing power and memory. They often demand far more resources ($C_{\text{model}} \gg C_{\text{limit}}$, $M_{\text{model}} \gg M_{\text{limit}}$) than our little edge devices can spare for either learning or day-to-day operation. What's more, trying to train a smart security model on a single device all by itself is a bit like asking a student to learn about the whole world by only ever seeing their own backyard. With such a limited set of local data to learn from ($\mathcal{D}_i$), the resulting AI model is likely to be a poor learner, unable to recognize new things and performing inconsistently across the entire network of devices (E). The only way forward is to have all the devices learn as a team, combining their collective experience to build a much smarter defense without ever having to share their private, local data.

Creating a map of interactions, what we technically call a graph ($G = (V, E, X)$), has proven to be a fantastic way to represent all the complex goings-on inside a computer, like how programs talk to files or connect to the internet. This method helps us spot behavioral patterns that would be completely invisible if we just looked at a simple list of events. The catch is that a computer's activity is never static; it's a constantly unfolding story. A regular graph-reading AI is good at looking at a single snapshot, but real-world behavior is more like a movie, a whole sequence of these snapshots ($G(t_1), G(t_2), \ldots, G(t_k)$). To understand this movie, you need a special kind of AI that can process graphs as they change over time. The big challenge is that running this kind of analysis in real-time on a small edge device is computationally very heavy. We need to create an AI architecture and data-handling techniques that are specifically built to be super lightweight and can keep up with new information as it happens. The tricky part is designing an AI that can effectively look back at the last few moments of activity (a recent history window, w) to understand the full context, all while staying within the strict power ($C_{\text{TGNN}} \leq C_{\text{limit}}$) and memory ($M_{\text{TGNN}} \leq M_{\text{limit}}$) budgets of the device it's running on.

To make matters worse, a growing problem is that attackers are now using AI's own tricks against it. They can make tiny, almost unnoticeable changes to the activity map–like adding a single fake road to a city plan–that can completely fool our security AI. A map that clearly showed a malicious attack ($G_{\text{malicious}}$) can be slightly tweaked ($G'_{\text{malicious}}$), and suddenly the AI is tricked into thinking everything is perfectly fine. Making sure our AI is tough enough to see through these clever disguises is absolutely critical if we want to trust it. Trying to solve this problem while also making the system work as a group (*federated*), understand the flow of time (*temporal*), and run on tiny (*lightweight*) devices is an incredibly complex puzzle that nobody has really cracked yet. We need a single, all-in-one framework that provides private, efficient, and real-time malware detection using these activity maps, while also making sure it's tough enough to resist these clever adversarial attacks, all within the strict limits of a scattered edge network. Specifically, the technical challenge we are tackling in this work is to design, build, and test a group-learning system that can train and use a lightweight, time-aware AI for real-time, deception-proof malware detection on resource-limited devices. This involves three key steps: First, figuring out the best way to draw these activity maps and build a lean AI that can read them without bogging down the device. Second, creating the rules for how the devices can study together, sharing only their key learnings to protect privacy and save on data usage. And third, weaving a special "boot camp" into this group learning process to train the final, shared AI to spot even the most cleverly disguised attacks, ensuring it can be trusted when things get serious.

2.2 Literature Review

The growing prevalence of distributed edge networks has amplified the need for efficient cybersecurity measures capable of operating within the unique constraints of this environment. Traditional malware detection techniques, predominantly reliant on static signatures or heuristic rules, have proven insufficient against the dynamic and increasingly evasive nature of modern malicious software [22]. While the adoption of machine learning (ML) and deep learning (DL) models has advanced detection capabilities, particularly for complex and previously unseen threats [1, 2, 23], as demonstrated by various hybrid approaches [3, 4, 9, 12, 24] and ensemble methods [6, 11, 13], centralized processing paradigms inherent to many of these approaches present significant scalability, latency, and privacy challenges when applied to the vast and geographically dispersed landscape of edge devices [19]. Transmitting the sheer volume of system telemetry data from numerous edge nodes to a central analytics platform for processing incurs substantial bandwidth costs and introduces unacceptable delays for real-time threat response [21], which is critical for mitigating damage from fast-propagating malware. The computational demands of complex DL models can also exceed the capabilities of many resource-constrained edge devices [17].

The intricate relationships and behavioral patterns exhibited by malware within a system or network can be effectively modeled as graph structures. Representing entities such as processes, files, and network connections as nodes, and their interactions as edges, provides a rich context for discerning malicious activity that may be obscured in simpler data formats [18]. Graph Neural Networks (GNNs) have emerged as powerful tools for learning representations and performing

classification directly on graph-structured data, demonstrating notable success in various cybersecurity applications, including network intrusion detection [25–27] and the analysis of static program properties for malware classification [7] nd even in related domains such as audio steganalysis [28]. However, existing GNN-based security solutions often primarily address static graph analysis or assume centralized processing capabilities. They frequently do not adequately account for the temporal evolution of system behaviors captured in dynamic graphs, the computational constraints of edge devices necessary for real-time inference, or the complexities introduced by a distributed operational environment [3].

Addressing the challenges of data privacy and centralized bottlenecks in distributed systems, Federated Learning (FL) has gained prominence as a paradigm enabling collaborative model training across decentralized devices without the need to pool raw data [19, 29–35]. FL has been applied to various security-related tasks, such as distributed intrusion detection [20] and collaborative threat intelligence based on feature vectors or time-series data [21]. While FL effectively preserves data privacy and reduces communication bandwidth compared to central data aggregation, its application to complex data structures like graphs, especially those that are dynamic and temporal, presents unique challenges [18]. Adapting FL for GNNs (Federated GNNs) requires addressing issues related to the heterogeneity of local graph data, the communication efficiency of transmitting graph-related model updates, and ensuring global model convergence and effectiveness across diverse local graph structures [20, 21]. Furthermore, the intersection of FL and adversarial robustness, while a growing area of research, has not been extensively explored in the specific context of defending against adversarial attacks on complex graph-structured data in resource-constrained edge environments.

A critical, yet often understated, challenge in deploying data-driven malware detection systems is their vulnerability to adversarial attacks. Adversaries can generate adversarial examples by introducing subtle perturbations to input data that are designed to mislead trained models, leading to misclassification and evasion [4]. In the context of GNNs, adversarial attacks can target graph structure, node features, or both, aiming to alter the model's decision boundaries [5, 7, 36–38]. While techniques for improving the adversarial robustness of GNNs through adversarial training or robust aggregation methods have been proposed [12, 13, 39–41], integrating these defenses effectively within a federated learning framework tailored for lightweight, temporal graph analysis on resource-constrained edge devices introduces significant technical complexities. The convergence of temporal graph processing, federated learning, lightweight model design,

The existing literature presents significant progress in individual domains of malware detection, graph neural networks, federated learning, and adversarial robustness. However, a notable gap exists in the unified application of these advancements to address the combined challenges of real-time, lightweight, adversarial-robust malware detection on dynamic graph data within the privacy-preserving and resource-constrained setting of distributed edge networks. Current solutions either lack the necessary real-time or lightweight capabilities for edge deployment, fail to handle the temporal dynamics of malware behavior effectively, do not incorporate robust defenses against adversarial evasion in a practical manner, or rely on centralized architectures unsuitable for distributed environments. This critical gap highlights the need for a novel framework capable of integrating these disparate requirements into a cohesive and deployable edge security solution.

2.3 Distinction from Prior Works

While recent works have explored Federated Learning (FL) for intrusion detection [19, 31] and Graph Neural Networks (GNNs) for security analytics [18, 27], EdgeFence introduces a uniquely integrated and edge-optimized framework that advances beyond the state-of-the-art in several fundamental ways. Unlike federated IDS approaches that rely on traditional feature vectors or time-series data, EdgeFence leverages *temporal graph representations* of system interactions, enabling richer modeling of dynamic attack behaviors. Compared to graph-based federated systems, which employ graph clustering for aggregation but do not model temporal evolution, EdgeFence incorporates a lightweight Temporal GNN (TGNN) capable of processing sequences of graph snapshots in real-time on resource-constrained devices. Furthermore, while adversarial robustness has been studied in centralized GNNs [39, 40], EdgeFence is the first to seamlessly integrate adversarial training *within* the federated learning cycle, hardening the global model against evasion attacks without compromising data privacy. This holistic co-design of federated learning, temporal graph learning, and adversarial defense–all under stringent edge resource constraints–distinguishes EdgeFence as a novel and comprehensive solution for malware detection in distributed edge networks.

3 Methodology

The architecture depicted in Fig 1 consists of a collection of dispersed edge nodes, each proficient in local data gathering and processing, with a central server tasked with coordinating the federated learning process. Each edge node perpetually observes system activities, producing dynamic,

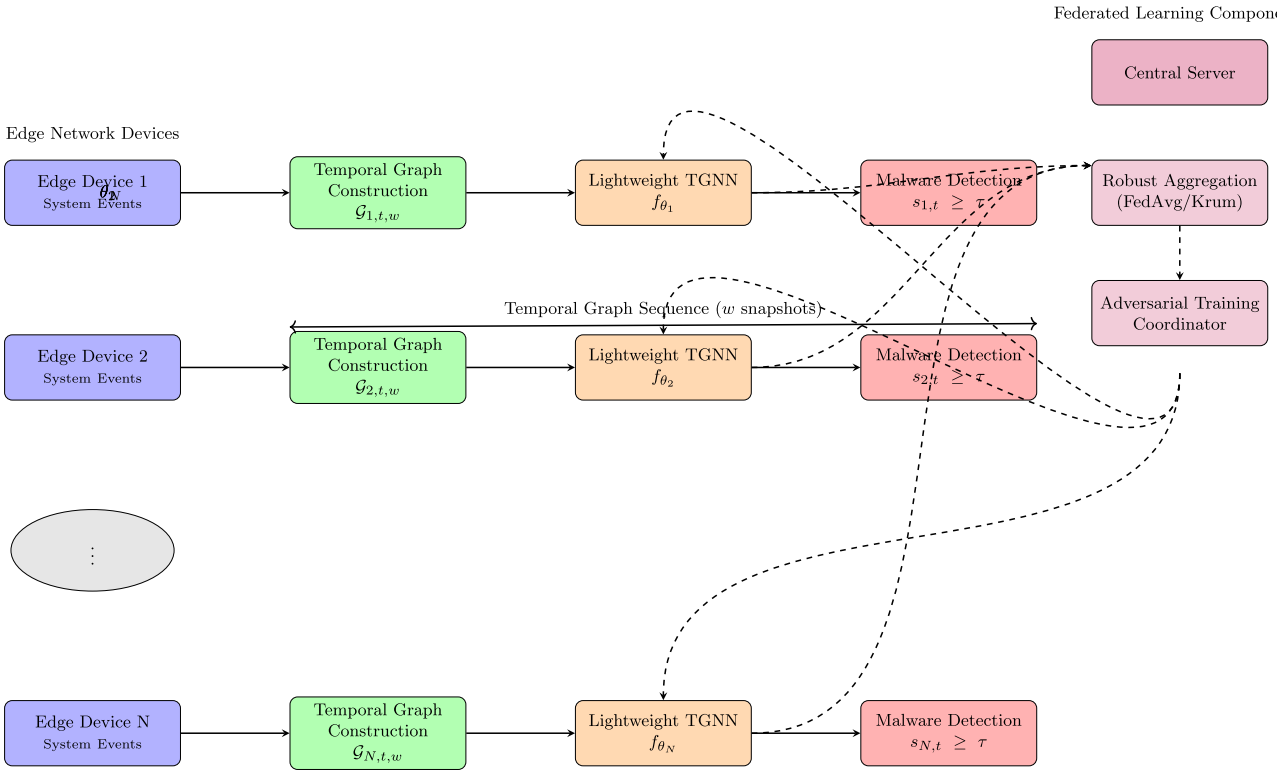


Fig. 1 Complete EdgeFence Architecture: Distributed Temporal GNN with Federated Learning and Adversarial Training

graph-structured representations of current processes, system calls, and network interactions in real-time. The local temporal graphs function as the input data for a lightweight TGNN model implemented on the edge device. The TGNN is engineered to effectively handle temporal graph streams, extracting characteristics that signify harmful activity with minimum computational cost and low latency, facilitating near real-time local inference, as outlined in Algorithm 2. EdgeFence utilizes a federated learning methodology to enhance collaborative learning and augment the generalization capabilities of the detection model while preserving data privacy. Edge nodes train their local TGNN models on the freshly obtained temporal graph data and securely broadcast only the model changes to the central server. The server consolidates these changes using a sophisticated aggregation algorithm to generate a new global model, which is subsequently disseminated to the edge nodes for implementation, as detailed in Algorithm 1. A fundamental aspect of EdgeFence's methodology is the use of adversarial training techniques throughout the federated learning process. This entails the local generation of adversarial instances from temporal graphs on edge devices and their integration into the local training process, with the possible utilization of robust aggregation techniques at the server. This method fortifies the TGNN models against hostile alterations, guaranteeing dependable detection despite evasion strategies. This section outlines the essential preliminaries for graph theory, graph neural networks, federated learning, and adversarial attacks to comprehensively grasp the technological foundations of the proposed EdgeFence system.

3.1 Preliminaries

This section provides the fundamental definitions, notations, and technical concepts related to graph theory, temporal graphs, Graph Neural Networks (GNNs), Temporal Graph Neural Networks (TGNNs), Federated Learning (FL), and adversarial attacks on graphs that form the technical basis for the design and analysis of the EdgeFence framework. Precision in these definitions is critical for understanding the subsequent technical contributions.

3.1.1 Graph Definition and Notation

A graph G is characterized as an ordered pair (V, E), where V denotes a finite collection of vertices, and E represents a set of edges that illustrate interactions between pairs of vertices. The cardinalities of these sets are represented as $|V| = n$ and $|E| = m$, respectively. Edges may be undirected, denoted as unordered pairs $\{u, v\}$, or directed, denoted as ordered pairs (u, v), signifying a connection from vertex u to vertex v. A simple graph is characterized by the absence

of loops (edges connecting a vertex to itself) and the prohibition of multiple edges between any two vertices. Graphs pertinent to system monitoring frequently exhibit characteristics related to nodes and/or edges. Node features are often denoted by a matrix $X \in \mathbb{R}^{n \times d}$, where d represents the dimensionality of the feature space for each node $v_i \in V$. Likewise, edge characteristics may be denoted by a matrix $E_f \in \mathbb{R}^{m \times d_e}$. The configuration of a graph is commonly shown by its adjacency matrix $A \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ indicates the presence of an edge from node v_i to node v_j, while $A_{ij} = 0$ signifies its absence. In undirected graphs, A exhibits symmetry. Weighted graphs enhance this concept by adding a weight w_{ij} to each edge, represented in a weighted adjacency matrix $W \in \mathbb{R}^{n \times n}$. In cybersecurity applications, graphs may be heterogeneous, comprising various types of nodes (e.g., processes, files, network sockets) and diverse sorts of edges (e.g., "reads", "writes", "connects_to"). A heterogeneous graph is characterized by vertex and edge type mappings $\phi_V : V \to \mathcal{T}_V$ and $\phi_E : E \to \mathcal{T}_E$, with $\mathcal{T}_V$ and $\mathcal{T}_E$ representing the collections of specified vertex and edge types, respectively. The neighborhood of a vertex v, represented as $\mathcal{N}(v)$, comprises the set of vertices that are nearby to v.

3.1.2 Temporal Graph Definition and Notation

Temporal graphs represent dynamic systems in which entities and their interactions change over time. In the realm of observing system activity on edge devices, we focus on temporal graphs as sequences of graph snapshots captured at distinct time intervals. A temporal graph is denoted as $\mathcal{G}_T = \{G_{t_1}, G_{t_2}, \ldots, G_{t_K}\}$, where $t_1 < t_2 < \cdots < t_K$ indicate the time steps, and each $G_{t_k} = (V_{t_k}, E_{t_k}, X_{t_k})$ constitutes a static graph snapshot that encapsulates the system state at time t_k. The node set V_{t_k}, edge set E_{t_k}, and node characteristics X_{t_k} may differ among snapshots. For example, V_{t_k} may encompass all processes operational at time t_k, E_{t_k} denotes their inter-process communication or file access events during the interval $[t_{k-1}, t_k)$, and X_{t_k} comprises attributes characterizing the status of each process. A continuous-time temporal graph represents events as discrete occurrences with timestamps $(u, v, \tau, \ldots)$, creating an edge stream. For analytical ease and compatibility with various TGNN architectures, the snapshot model across a pertinent temporal window w, represented as $\{G_{t-w+1}, \ldots, G_t\}$, is frequently utilized to record current dynamic behavior.

3.1.3 Graph Neural Networks (GNNs)

Graph Neural Networks are a category of deep learning models intended to analyze graph-structured data by systematically collecting information from node neighborhoods to acquire sophisticated representations at the node, edge, or graph level. A standard GNN functions via a message-passing protocol over L levels. At layer $l \in \{1, \ldots, L\}$, each node v modifies its feature representation $h_v^{(l)}$ utilizing the representations of its neighbors from the preceding layer $h_u^{(l-1)}$ and its own prior representation $h_v^{(l-1)}$. The process may be encapsulated by the subsequent two functions:

$$a_v^{(l)} = \text{AGGREGATE}^{(l)}(\{h_u^{(l-1)} \mid u \in \mathcal{N}(v) \cup \{v\}\})$$

$$h_v^{(l)} = \text{COMBINE}^{(l)}(h_v^{(l-1)}, a_v^{(l)})$$

where $\text{AGGREGATE}^{(l)}$ is invariant to the permutation of neighbors (e.g., sum, mean, max, or more sophisticated permutation-invariant neural networks) and $\text{COMBINE}^{(l)}$ integrates the aggregated information with the node's prior state (e.g., concatenation followed by a linear transformation and activation function, or gated mechanisms). Various GNN versions emerge from certain selections of AGGREGATE and COMBINE functions. For instance, Graph Convolutional Networks (GCNs) employ a distinct aggregation method grounded in the graph Laplacian, whereas Graph Attention Networks (GATs) utilize an attention mechanism to provide varying weights to neighboring nodes during aggregation. $a_v^{(l)} = \sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} W^{(l)} h_u^{(l-1)}$, where $\alpha_{vu}^{(l)}$ denotes the attention coefficients. Subsequent to L layers, the ultimate node embeddings $h_v^{(L)}$ encapsulate structural and feature information from the node's L-hop vicinity. In graph-level tasks such as classification, a READOUT function is utilized to consolidate the final node embeddings into a singular graph embedding $h_G = \text{READOUT}(\{h_v^{(L)} \mid v \in V\})$, employing methods such global pooling (sum, mean, max) or attention-based pooling.

3.1.4 Temporal Graph Neural Networks (TGNNs)

Temporal Graph Neural Networks are a step up from regular GNNs because they are built to understand situations that change over time. They don't just see how things are connected at one specific moment; they also grasp the "story" of how those connections evolve. How they add this understanding of time can be done in a couple of different ways. A very common approach is to look at a series of snapshots in time, like flipping through the pages of a photo album $\{G_{t-w+1}, \ldots, G_t\}$. A regular GNN is used on each individual "photo" to understand that specific moment. Then, the results from all the photos are fed into a different kind of AI, like a Recurrent Neural Network (RNN), which is really good at understanding sequences. This helps it piece together the whole story across all the snapshots [42]. Another way is to build the concept of time directly into the GNN's main thought process. This could mean having the AI pay more attention to newer information, or stamping every piece of data with a "time code" before it even starts its analysis [43].

When we're designing these for small edge devices, there's a tricky balance to strike. They have to be smart enough to find important patterns but also be extremely efficient. This often means we have to simplify things, maybe by having the AI look at only a sample of neighbors instead of all of them, using simpler math, or building it in a way that it can handle small updates without having to re-process the entire history every time [44].

3.1.5 Federated Learning (FL)

Federated Learning is a team-based approach to training a shared AI model, which we can call θ. A central server coordinates everything, but the most important rule is that the raw training data never leaves each user's device. The process works in a series of communication rounds, labeled as $r = 1, 2, \ldots$. Here's what happens in each round:

1. The central server starts by sending the current version of the shared model, $\theta^{(r)}$, to a select group of devices, $\mathcal{S}^{(r)} \subseteq \{1, \ldots, N\}$, where N is the total number of devices in the network.
2. Every device that was selected, $i \in \mathcal{S}^{(r)}$, then takes this model and trains it privately on its own local data, $\mathcal{D}_i$. The goal is to improve the model by minimizing its errors on this local data, $\mathcal{L}_i(\theta)$. It starts with the model it was given, $\theta^{(r)}$, and after its private training, it ends up with an improved set of local parameters, $\theta_i^{(r+1)}$.
3. The devices in the group $\mathcal{S}^{(r)}$ then send their improvements back to the server. They don't send the data itself, but rather just the "lessons learned." This is often either the direct change they made, $\Delta\theta_i^{(r)} = \theta_i^{(r+1)} - \theta^{(r)}$, or the raw learning directions (gradients), $\nabla\mathcal{L}_i(\theta^{(r)})$.
4. The server takes all the updates it received and combines them to create the next version of the global model, $\theta^{(r+1)}$. A popular method for this is called Federated Averaging (FedAvg), where the new global model becomes a weighted average of all the local improvements: $\theta^{(r+1)} \leftarrow \sum_{i \in \mathcal{S}^{(r)}} \frac{|\mathcal{D}_i|}{\sum_{j \in \mathcal{S}^{(r)}} |\mathcal{D}_j|} \theta_i^{(r+1)}$.

This cycle of sending, training, and combining continues until the shared model is good enough.

3.1.6 Adversarial Attacks on Graphs

Adversarial attacks on GNNs try to construct perturbed graphs $G' = (V, E', X')$ that are marginally different from a clean graph $G = (V, E, X)$ but cause a target GNN model f_θ to deliver an inaccurate output for G' [45]. For malware detection, the primary goal of an adversary is often evasion: given a malicious graph $G_{malicious}$, find a perturbation $\delta = (\delta_A, \delta_X)$ such that the perturbed graph $G'_{malicious} = (V, A + \delta_A, X + \delta_X)$ is classified as benign by the model, i.e., $f_\theta(G'_{malicious}) = \text{Benign}$, while $f_\theta(G_{malicious}) = \text{Malicious}$. The perturbation δ must comply to a threat model, which constrains the types and magnitudes of allowable alterations to maintain realism and stealthiness. Common threat models allow a limited number of edge additions or deletions ($||\delta_A||_0 \leq k$), bounded changes to node features ($||\delta_X||_p \leq \epsilon$), or structural constraints like maintaining graph connectivity or degree distributions. Attack strategies may be classed into white-box attacks (where the adversary has complete knowledge of the model parameters and architecture) and black-box attacks (where the adversary can only query the model).

3.2 EdgeFence Framework

The framework comprises several key technical components operating collaboratively across distributed edge nodes and a central server, as detailed in the following sub-subsections.

3.2.1 Local Temporal Graph Construction and Representation

In our approach, we consider a system with multiple edge devices, indexed by $i \in \{1, \ldots, N\}$. We utilize benchmark datasets (as described in Section 4.1) where each instance provides records that we process to simulate fine-grained, system-level events. We construct temporal graph sequences from these dataset instances for model training and evaluation. Specifically, discrete events within each instance are collected over fixed time intervals ΔT to generate a sequence of dynamic graph snapshots. For a time step t, we construct the local graph $G_{i,t} = (V_{i,t}, E_{i,t}, X_{i,t})$. Here, the node set $V_{i,t}$ comprises active entities observed during the interval $[t - \Delta T, t)$, such as process identifiers (PIDs), file descriptors, or network socket tuples (source IP/port, destination IP/port, protocol). The edge set $E_{i,t}$ represents interactions between these entities within the same interval; for example, $(p_1, f_1) \in E_{i,t}$ if process p_1 accesses file f_1. Node features $X_{i,t} \in \mathbb{R}^{|V_{i,t}| \times d_v}$ capture relevant attributes of the entities, such as process name, user ID, execution path, file permissions, or network connection state. To capture temporal dependencies, the input to the local TGNN at time t is a sequence of the most recent w graph snapshots: $\mathcal{G}_{i,t,w} = \{G_{i,t-w\Delta T+\Delta T}, \ldots, G_{i,t}\}$.

3.2.2 Lightweight Temporal Graph Neural Network Architecture

The primary detection engine in our architecture is a streamlined Temporal Graph Neural Network, referred to as f_θ. This TGNN analyzes the series of local graph snapshots $\mathcal{G}_{i,t,w}$ and produces a maliciousness score or categorization. The design

reconciles the necessity to record intricate spatio-temporal patterns with the stringent computational and memory limitations of edge devices. It utilizes a streamlined multi-layer GNN architecture for each graph snapshot in the series, succeeded by a temporal aggregation process. For each snapshot $G_{i,t'} \in \mathcal{G}_{i,t,w}$, node embeddings are calculated utilizing L GNN layers: $H_{i,t'}^{(l)} = \text{GNNLayer}^{(l)}(A_{i,t'}, H_{i,t'}^{(l-1)})$, with $H_{i,t'}^{(0)} = X_{i,t'}$. The AGGREGATE and COMBINE functions in $\text{GNNLayer}^{(l)}$ are selected for efficiency, including mean aggregation and straightforward linear transformations with non-linear activations $\sigma(\cdot)$ (e.g., ReLU). Upon completion of L layers, a snapshot-level representation is acquired, either using a global readout function $h_{G_{i,t'}} = \text{READOUT}(H_{i,t'}^{(L)})$ or by preserving node-level representations. The snapshot representations $\{h_{G_{i,t-w\Delta T+\Delta T}}, \ldots, h_{G_{i,t}}\}$ (or sequences of node embeddings) are subsequently input into a streamlined temporal processing module, such as a single-layer GRU or a temporal attention mechanism with a predetermined history length, to generate a final temporal embedding $\mathbf{h}_{i,t}^{temp}$. A concluding classification layer, including a dense layer succeeded by a sigmoid activation for binary classification, translates $\mathbf{h}_{i,t}^{temp}$ into a maliciousness score $s_{i,t} \in [0, 1]$. The lightweight architecture is accomplished by reducing the number of GNN layers L, using efficient aggregation methods, constraining the sizes of hidden dimensions, and improving the temporal processing module for minimal computing impact.

3.2.3 On-Device Real-Time Inference

After the global TGNN model $f_{\theta_{global}}$ has been trained by Federated Learning and implemented on the edge device i, we conduct real-time malware detection simulations. At each time step t, the local graph creation module produces the most recent snapshot $G_{i,t}$. The new snapshot is included into the temporal window $\mathcal{G}_{i,t,w}$, perhaps necessitating efficient changes to node and edge sets as well as features. The lightweight TGNN $f_{\theta_{global}}$ subsequently analyzes $\mathcal{G}_{i,t,w}$ to get the maliciousness score $s_{i,t}$. A threshold τ_{detect} is imposed on $s_{i,t}$: if $s_{i,t} \geq \tau_{detect}$, an alarm is activated, signifying possible malware activity. The essential condition for real-time operation is that the inference time $\Delta t_{inference}$ for processing $\mathcal{G}_{i,t,w}$ and calculating $s_{i,t}$ must be much less than the snapshot interval ΔT, namely, $\Delta t_{inference} \ll \Delta T$. The TGNN's lightweight design and improved graph processing algorithms are essential for achieving low-latency performance on resource-constrained hardware.

3.2.4 Federated Training Protocol for TGNNs

EdgeFence employs a federated learning protocol to collaboratively train the lightweight TGNN model f_θ across the distributed edge nodes without centralizing sensitive local graph data. The process proceeds in rounds $r = 1, 2, \ldots$, orchestrated by a central server.

1. *Global Model Distribution*: The server broadcasts the current global model parameters $\theta^{(r)}$ to a selected subset of edge clients $\mathcal{S}^{(r)} \subseteq \{1, \ldots, N\}$.
2. *Local Model Training*: Each selected client $i \in \mathcal{S}^{(r)}$ trains its local TGNN model parameters θ_i starting from $\theta^{(r)}$ using its private local dataset $\mathcal{D}_i$. The local dataset $\mathcal{D}_i = \{(\mathcal{G}_{i,t,w}^{(j)}, y_i^{(j)})\}_{j=1}^{|\mathcal{D}_i|}$ consists of sequences of temporal graph snapshots $\mathcal{G}_{i,t,w}^{(j)}$ and their corresponding ground truth labels $y_i^{(j)} \in \{0, 1\}$ (benign/malicious), collected over time. Local training minimizes the empirical risk $\mathcal{L}_i(\theta) = \frac{1}{|\mathcal{D}_i|} \sum_{(\mathcal{G}_T, y) \in \mathcal{D}_i} \ell(f_\theta(\mathcal{G}_T), y)$, where $\ell(\cdot, \cdot)$ is the loss function (e.g., binary cross-entropy). This is typically done using a local optimizer (e.g., SGD, Adam) for E_{local} epochs. The updated local model parameters are $\theta_i^{(r+1)}$.
   item *Local Update Upload*: Clients in $\mathcal{S}^{(r)}$ securely transmit their local model updates, often $\Delta\theta_i^{(r)} = \theta_i^{(r+1)} - \theta^{(r)}$, to the central server. The TGNN's compact design (80KB) keeps each update small, minimizing per-round communication overhead. With a lightweight TGNN model containing approximately ∼80 KB of parameters, each client uploads about approximately 80 KB per round when sending the full parameter update $\Delta\theta_i^{(r)}$. Given R federated rounds and a client participation rate of S_{rate}, the total upstream communication volume is roughly $R \times (S_{rate} \cdot N) \times 80$ KB.
3. *Global Model Aggregation*: The server aggregates the received updates from the selected clients. EdgeFence utilizes weighted average aggregation algorithm (e.g., FedAvg: $\theta^{(r+1)} \leftarrow \sum_{i \in \mathcal{S}^{(r)}} \frac{|\mathcal{D}_i|}{\sum_{j \in \mathcal{S}^{(r)}} |\mathcal{D}_j|} \theta_i^{(r+1)}$).

We adopt this iterative process to allows the global model to learn from the diverse malware behaviors encountered across the distributed edge network while preserving the privacy of the local graph data.

3.2.5 Adversarial Robustness via Federated Adversarial Training

To ensure the detection model's resilience against adversarial evasion attacks, EdgeFence integrates adversarial training within the federated learning framework. We perform this during the local training phase on each edge device. For each training sample $(\mathcal{G}_{i,t,w}, y_i) \in \mathcal{D}_i$, the client generates an adversarial version $\mathcal{G}'_{i,t,w}$ by applying a perturbation δ to the graph snapshots within the sequence, i.e., $G'_{i,t'} = G_{i,t'} + \delta_{t'}$ for $t' \in \{t - w\Delta T + \Delta T, \ldots, t\}$, where $\delta = \{\delta_{t'}\}$ is constrained by a predefined threat model Δ.

This involves finding δ that approximately maximizes the local loss $\mathcal{L}_i(f_{\theta_i}(\mathcal{G}_{i,t,w} + \delta), y_i)$. Given the resource constraints, we use a lightweight adversarial example generation method locally, such as a single-step gradient ascent attack or a computationally efficient projected gradient descent (PGD) approximation on a continuous relaxation of the graph structure and features. The local model is then trained to minimize a loss function that penalizes errors on both clean and these locally generated adversarial examples. The objective for client i becomes approximately minimizing $\mathbb{E}_{(\mathcal{G}_T,y)\sim\mathcal{D}_i}\left[\ell(f_{\theta_i}(\mathcal{G}_T), y)+\lambda\cdot\ell(f_{\theta_i}(\mathcal{G}_T+\delta^*(\mathcal{G}_T,\theta_i)), y)\right]$, where δ^* is the adversarial perturbation found for $(\mathcal{G}_T, y)$ and λ is a weighting factor. Incorporating these adversarial examples into the local training of each client's TGNN allows the model to learn robustness against diverse attack strategies encountered across the distributed network, which is then aggregated into the global model parameters $\theta^{(r+1)}$ during the server aggregation step.

3.2.6 Lightweight Design Considerations

It is essential to achieve a lightweight design for the deployment of EdgeFence on edge devices with limited resources. This is addressed throughout the framework. Initially, the construction of the temporal graph emphasises pertinent system events and may utilise graph reduction methods (such as filtering out low-importance nodes and edges) to maintain a manageable graph size. Secondly, the TGNN architecture is crafted with a limited number of layers, streamlined hidden dimensions, and efficient aggregation and combination functions. Thirdly, the chosen method for generating local adversarial examples is designed to be computationally lightweight, enabling efficient operation on edge hardware throughout the local training phases. Ultimately, the communication in federated learning is restricted to the transmission of model updates, which are considerably smaller than raw data. This approach effectively reduces bandwidth usage and power consumption linked to network communication. Striking a balance among detection performance, adversarial robustness, and resource utilisation is a crucial optimisation goal in the design and training of the EdgeFence framework.

4 Implementation

This section provides specific technical details regarding the implementation choices we made for each component within this simulation environment, adhering to the architectural principles and leveraging the preliminary concepts previously described. We conducted our evaluation using pre-collected, real-world edge-network malware datasets to drive the simulated edge client activities, deferring live deployment case studies to future work.

Algorithm 1 EdgeFence Federated Temporal Graph Neural Network Training

Require: Global model parameters $\theta^{(0)}$, number of federated rounds R, number of local epochs E_{local}, learning rate η, batch size B, client fraction S_{rate}, adversarial weighting factor λ, PGD steps K_{attack}, edge modification budget k_{mod}, feature perturbation bound ϵ.

Initialize central server with global model $f_{\theta^{(0)}}$.
Distribute initial global model parameters $\theta^{(0)}$ to all N edge clients.
for each federated round $r = 1, \ldots, R$ **do**
  Server selects a subset of active clients $\mathcal{S}^{(r)} \subseteq \{1, \ldots, N\}$ of size $S_{rate} \cdot N$.
  Server broadcasts current global model parameters $\theta^{(r)}$ to all clients in $\mathcal{S}^{(r)}$.
  for each client $i \in \mathcal{S}^{(r)}$ **in parallel do**
    Client i initializes its local model f_{θ_i} with $\theta^{(r)}$.
    Client i accesses its private local dataset $\mathcal{D}_i = \{(\mathcal{G}_{i,t,w}^{(j)}, y_i^{(j)})\}_{j=1}^{|\mathcal{D}_i|}$.
    Define local loss function $\ell(\cdot,\cdot)$ (e.g., weighted Binary Cross-Entropy).
    Define local optimizer (e.g., Adam with learning rate η).
    for each local epoch $e = 1, \ldots, E_{local}$ **do**
      for each mini-batch $(\mathcal{G}_T, y) \in \mathcal{D}_i$ **do**
        // Adversarial Example Generation
        Generate adversarial perturbation δ^* using a lightweight PGD variant:
        $\delta^* \leftarrow \arg\max_{\|\delta\|\le\Delta} \ell(f_{\theta_i}(\mathcal{G}_T + \delta), y)$ for K_{attack} steps,
        where Δ constrains edge modifications (k_{mod}) and feature perturbations (ϵ).
        $\mathcal{G}'_T \leftarrow \mathcal{G}_T + \delta^*$ // Adversarial graph sequence
        // Local Model Training with Adversarial Regularization
        Compute composite loss: $\mathcal{L}_i^{adv} = \ell(f_{\theta_i}(\mathcal{G}_T), y) + \lambda \cdot \ell(f_{\theta_i}(\mathcal{G}'_T), y)$
        Perform a gradient descent step to minimize $\mathcal{L}_i^{adv}$ with respect to θ_i.
      end for
    end for
    Client i obtains updated local model parameters $\theta_i^{(r+1)}$.
    Client i computes local update $\Delta\theta_i^{(r)} = \theta_i^{(r+1)} - \theta^{(r)}$.
    Client i securely transmits $\Delta\theta_i^{(r)}$ (or $\theta_i^{(r+1)}$) and $|\mathcal{D}_i|$ to the server.
  end for
  Server aggregates received updates:
  $\theta^{(r+1)} \leftarrow \sum_{i\in\mathcal{S}^{(r)}} \frac{|\mathcal{D}_i|}{\sum_{j\in\mathcal{S}^{(r)}}|\mathcal{D}_j|}\theta_i^{(r+1)}$ (FedAvg aggregation).
end for
return Final global model parameters $\theta^{(R)}$.

Algorithm 2 EdgeFence On-Device Real-Time Inference

Require: Deployed global model $f_{\theta_{global}}$, snapshot interval ΔT, temporal window size w, detection threshold τ_{detect}.

Edge device continuously monitors system-level events.
for each time step t **do**
  // Local Temporal Graph Construction
  Collect discrete events within interval $[t - \Delta T, t)$.
  Construct new local graph snapshot $G_{i,t} = (V_{i,t}, E_{i,t}, X_{i,t})$.
  Update temporal window: $\mathcal{G}_{i,t,w} \leftarrow \{G_{i,t-w\Delta T+\Delta T}, \ldots, G_{i,t}\}$.
  Ensure efficient updates to $V_{i,t}, E_{i,t}, X_{i,t}$ to maintain low latency.
  // Lightweight TGNN Inference
  Compute maliciousness score: $s_{i,t} \leftarrow f_{\theta_{global}}(\mathcal{G}_{i,t,w})$.
  Measure inference time $\Delta t_{inference}$.
  Assert: $\Delta t_{inference} \ll \Delta T$.
  // Anomaly Detection
  if $s_{i,t} \ge \tau_{detect}$ **then**
    Trigger an alert (e.g., indicate potential malware activity).
  end if
end for

4.1 Data Simulation and Local Temporal Graph Construction Implementation

We utilize event streams extracted from the CIC-IDS[22] and IoT-23[47] datasets in this experimental implementation with each samples about 3.110 millions and 3.0 millions samples respectively. These datasets represent traffic and system logs from edge-like environments under various attack scenarios,

providing the raw material for simulating the data generation process on each simulated edge client $i \in \{1, \ldots, N\}$. It is important to note that both datasets are characterized by a very high class imbalance, where benign traffic constitutes the the significant majority of samples, and malicious instances represent minority class with a ratio of 7:3.

The extracted events, structured as tuples $(timestamp, event_type, process_id, \ldots)$, are processed chronologically for each simulated client. Every ΔT (set to 1 second in our experiments to simulate near real-time conditions), we construct a graph snapshot $G_{i,t} = (V_{i,t}, E_{i,t}, X_{i,t})$ from the events pertaining to client i that occurred within the interval $[t - \Delta T, t)$. Nodes $V_{i,t}$ are entities active during this period, identified by parsed event fields (e.g., unique process IDs derived from logs, file paths, network connection tuples). Edges $E_{i,t}$ represent recorded interactions between these entities as logged in the datasets (e.g., a log entry indicating a process created a file translates to a directed edge). Node features $X_{i,t}$ are extracted or aggregated from available event metadata within the datasets (e.g., process command line from CIC-IDS, or connection state from IoT-23). The sequence of temporal graph snapshots $\mathcal{G}_{i,t,w} = \{G_{i,t-w\Delta T+\Delta T}, \ldots, G_{i,t}\}$ is then formed by maintaining a window of the last w snapshots (e.g., $w = 10$, covering 10 seconds of simulated history) for each simulated client. Each temporal graph window $\mathcal{G}_{i,t,w}$ is assigned a binary label $y_{i,t} \in \{0, 1\}$ corresponding to the ground truth of the most recent snapshot $G_{i,t}$ within the window. This ensures a causal, real-time detection setting where the model must classify the current system state based only on historical observations. No future information (i.e., snapshots beyond time t) is used in constructing or labeling $\mathcal{G}_{i,t,w}$, thereby eliminating any risk of temporal leakage during training or evaluation.

4.2 Lightweight Temporal GNN Architecture Implementation Details

The Lightweight Temporal GNN f_θ examined in this work is developed using TensorFlow, a library that supports the building of GNNs. The GNN module of the TGNN is designed with $L = 3$ layers to optimize representational capacity and computational efficiency. We utilize a GraphSAGE-inspired design [48] with mean aggregation, which provides advantageous scalability characteristics: $h_v^{(l)} = \text{ReLU}(W_{\text{self}}^{(l)} h_v^{(l-1)} + W_{\text{neigh}}^{(l)} \text{MEAN}_{u \in \mathcal{N}(v)}(h_u^{(l-1)}))$. The node embedding dimension d_h is uniformly established at 32 throughout all GNN layers. Subsequent to the GNN layers, we implement global mean pooling to derive snapshot-level representations. $h_{G_{i,t'}} = \text{MEAN}_{v \in V_{i,t'}} h_v^{(3,t')}$. The temporal module is executed as a single-layer GRU including 64 hidden units, analyzing the sequence $\{h_{G_{i,t-w\Delta T+\Delta T}}, \ldots, h_{G_{i,t}}\}$. The concluding classification layer is a linear layer that transforms the GRU's final hidden state into a singular output neuron, succeeded by a sigmoid activation function. The total count of trainable parameters $|\theta|$ for this setup is roughly 79.63 KB, intentionally designed to emulate a lightweight model appropriate for deployment on resource-constrained edge hardware, projected to be less than 1 MB when saved in conventional model formats.

4.3 Federated Training Protocol and Parameter Settings

We simulate the Federated Learning process using a custom Python implementation mimicking the central server and client interactions. We implement the standard FedAvg algorithm for global model aggregation. In each round r, we select a random subset of simulated clients, comprising $S_{rate} \cdot N$ clients (where $S_{rate} = 0.1$, simulating selection of 10% of active edge devices per round). The total number of federated learning rounds R is set to 100, determined empirically based on convergence behavior observed on a separate validation portion of the datasets. The local optimizer is Adam with an initial learning rate $\eta = 0.001$, decaying by a factor of 0.5 every 100 rounds. We use default Adam parameters ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-7}$). Local training batches consist of $B = 64$ sequences of temporal graphs. The loss function $\ell(\cdot, \cdot)$ is Binary Cross-Entropy (BCE), as defined previously. For the issue of class imbalance, we utilize a weighted BCE to mitigate bias towards the majority class. Clients transmit the full local model parameters $\theta_i^{(r+1)}$ to the simulated server for aggregation.

4.4 Adversarial Training Implementation Details

Adversarial training is integrated into the local training loop simulated on each client. For each mini-batch of clean temporal graph samples $\{(\mathcal{G}_T^{(j)}, y^{(j)})\}_{j=1}^B$, we generate corresponding adversarial samples $\{\mathcal{G}_T^{'(j)}\}_{j=1}^B$. We use a lightweight adversarial attack method for on-device generation, implementing a variant of Projected Gradient Descent (PGD) tailored for graph data, executed for $K_{attack} = 3$ steps. This lightweight method is chosen to reflect the computational feasibility on resource-constrained edge devices in a real deployment. The attack perturbs the adjacency matrices and node features of the graphs within the temporal sequence $\mathcal{G}_{i,t,w}^{(j)}$ to maximize the local loss $\ell(f_{\theta_i}(\mathcal{G}_T^{(j)} + \delta^{(k)}), y^{(j)})$, projecting the perturbations back into the allowed threat model Δ after each step. The threat model Δ for the perturbation δ is defined by restricting the number of edge modifications (additions or deletions) to at most $k = 3$ per snapshot in the sequence and bounding the L_∞ norm of feature perturbations to $\epsilon = 0.05$. The local model is trained to minimize a com-

Table 1 Feature Selection for Network Anomaly Detection in IoT-23 and CIC-IDS Datasets, Motivated by Prior Research on Flow-Based and Behavioral Detection [25, 26, 46]

Category	IoT-23 Dataset Features	CIC-IDS Dataset Features
Flow Statistics	duration, timestamp	Flow Duration, timestamp, Sourc_Port, Destination_Port
	orig_bytes, resp_bytes	Total Length of Fwd/Bwd Packet
	orig_pkts, resp_pkts	Total Fwd/Bwd Packets
	orig_ip_bytes, resp_ip_bytes	Flow Bytes/s, Flow Packets/s
Packet-Level Features	missed_bytes	Fwd/Bwd Packet Length (Max, Min, Mean)
		Packet Length Variance
		Fwd/Bwd Header Length
Protocol Indicators	proto_tcp, proto_udp	Protocol (implicit in CIC-IDS)
		ICMP Type/Code (optional)
Connection States	conn_state_S0, SF, REJ	SYN Flag Count, RST Flag Count
	conn_state_RSTO, RSTR	ACK Flag Count, PSH Flag Count
		Fwd/Bwd URG/PSH Flags
Behavioral Features		Fwd/Bwd Bytes/Bulk Avg
		Subflow Fwd/Bwd Bytes
		Fwd/Bwd Init Win Bytes

posite loss $\mathcal{L}_i^{adv}(\theta_i)$, combining the BCE loss on clean and adversarial examples with a weighting factor $\lambda = 0.8$ for the adversarial term. This adversarial training process is computationally managed to stay within the simulated resource limits of the edge device during the training phase.

4.5 Detection Threshold and Performance Evaluation

Upon completion of the federated training, the final global model θ_{global} is evaluated on an unseen test portion of the CIC-IDS [22] and IoT-23 [47] datasets. We compute the maliciousness score $s_{i,t}$ for each temporal graph sequence $\mathcal{G}_{i,t,w}$ in the test set. The detection threshold τ_{detect} is determined based on the desired False Positive Rate (FPR) on a separate validation subset; specifically, we set τ_{detect} to achieve an FPR of 0.1% on the validation data, and performance metrics are then reported at this operating point on the test set. We evaluate performance using standard classification metrics: Accuracy, Precision, Recall, F1-score, and AUC. We assess the lightweight nature by measuring the simulated inference time $\Delta t_{inference}$, memory usage $M_{inference}$ (model size), and estimated CPU utilization $C_{inference}$ of the trained model on representative edge hardware specifications. Adversarial robustness is quantified by generating adversarial versions of the test set using various attack methods (including the PGD variant used in training) under different perturbation budgets and reporting the detection scores on these perturbed datasets. We monitor the scalability and convergence of the federated learning approach by analyzing the global model's test accuracy over FL rounds with varying numbers of simulated clients.

4.6 Experimental Setup and Environment

We conduct the evaluation within a controlled simulation environment. Edge clients are simulated as independent processes or threads, each allocated a partition of the preprocessed CIC-IDS and IoT-23 samples with their features presented in Table 1 and all implementation pecifications are presented in Table 2. We partition the datasets across the simulated clients to mimic a non-IID data distribution, where each client's partition contains events primarily from a specific set of machines or network segments within the original dataset. The central server is simulated as a separate process that manages client selection, model aggregation, and global model distribution. Communication between simulated clients and the server is implemented via standard network protocols (e.g., TCP/IP), simulating realistic network latency and bandwidth constraints, although the actual data transferred is limited to model updates as per the FL protocol. Experiments are conducted on a computing cluster. The primary machine specifications for the simulations include an NVIDIA GeForce RTX 4070 Super with 16GB of GPU memory, an Intel64 processor clocked at 3.2 GHz, and 32GB of system memory. The experimental framework records key metrics such as local inference time, computation overhead such memory usage and CPU usage, and final detection performance under various configurations, including different numbers of clients.

Table 2 Core Implementation Specifications

Parameter	Value / Description
Random Seed	42
Federated Rounds (R)	100
Total Clients (N)	100
Client Participation Rate	10%
Snapshot Interval (ΔT)	1 second
Temporal Window (w)	10
Batch Size	64
Learning Rate	0.001
Model Size	∼80 KB (float32)
GNN Layers (L)	3
Hidden Dimension (d_h)	32
Adversarial Steps (K_{attack})	3
Perturbation Budget (ϵ)	0.05 (L_∞)
Max Edge Modifications (k)	3 per snapshot
Adversarial Weight (λ)	0.8
Detection Threshold (τ_{detect})	FPR = 0.1%
Datasets	CIC-IDS2018, IoT-23
Class Ratio (Benign:Malicious)	7:3

4.7 Results & Analysis

Figure 2 illustrates the confusion matrix outcomes on untainted data for the CIC-IDS and IoT-23 datasets, emphasizing the baseline efficacy of the EdgeFence concept in non-adversarial circumstances. The conceptual structure of these matrices clearly reveals a significant concentration of accurately categorized examples, resulting in high values in the True Positive (TP) and True Negative (TN) quadrants. This empirical discovery indicates that EdgeFence is proficient in recognizing normal network traffic and, importantly, in precisely detecting hostile actions. The low False Positive (FP) rate, an essential statistic for practical applications, is also notably evident. This demonstrates the framework's strong ability to prevent the erroneous identification of benign system behaviors or network flows as malicious, which is essential for avoiding unnecessary resource usage, reducing alert fatigue for system administrators, and ensuring the continuous operation of resource-limited edge devices. Simultaneously, the persistently low False Negative (FN) rate across both datasets unequivocally illustrates the system's capacity to reliably identify authentic threats, hence substantially diminishing the probability of malware eluding detection and remaining within the system. These results unambiguously validate the temporal graph neural network (TGNN) architecture's capacity to capture patterns of dynamic processes and system interactions. The TGNN component adeptly models temporal dependencies in network flows and system calls, allowing it to differentiate between normal behavioral sequences and the nuanced, time-evolving abnormal or malevolent patterns characteristic of malware. The consistently high performance exhibited across the diverse CIC-IDS and IoT-23 datasets further emphasizes EdgeFence's robust generalization abilities, illustrating its relevance in various edge environments where traffic patterns and attack vectors can differ markedly according to application domain, device type, and communication protocols. This baseline validation is essential, demonstrating the fundamental correctness of the detection mechanism prior to the incorporation of hostile defensive complications.

Figure 3 highlights EdgeFence's performance under adversarial situations, offering essential insights into its resistance against complex attacks. The confusion matrices for the CIC-IDS and IoT-23 datasets in Figure 3 indicate a little decline in performance relative to the clean data findings in Figure 2; nevertheless, this decrease is notably limited. The True Positive (TP) and True Negative (TN) values are notably high, whereas the False Positive (FP) and False Negative (FN) rates exhibit relatively minor increases. This result clearly confirms the integration of adversarial training inside the federated learning framework. The Adversarial Training module, utilized during local client updates in the Federated Learning process, is crucial for this increased robustness. This module produces gradient-based adversarial perturbations, employing adjustable parameters like 'epsilon' to regulate perturbation intensity, 'iterations' to enhance adversarial example generation. These simulated adversarial manipulations necessitate that each client model cultivates increasingly strong and durable internal representations of both harmful and benign patterns. Thus, the global model, created by the weighted amalgamation of numerous resilient local updates, possesses an enhanced protection against various evasion tactics. The persistently low false negative rate under adversarial attack, as strongly illustrated in Figure 3, is very significant. This suggests that the model retains its ability to detect subtle or modified variants of known malware, a skill that is becoming increasingly vital due to the rise of polymorphic, disguised, and zero-day attacks. The consistency of detection performance in adversarial contexts highlights the intrinsic benefit of decentralizing training among diverse clients in the federated learning framework, which fosters a more generalized decision boundary and mitigates overfitting, thus rendering the global model less vulnerable to targeted adversarial attacks.

Figure 4 illustrates the synthesis of EdgeFence's full detection capabilities by offering a direct metric-wise comparison of model performance on clean and adversarial data. Figure 4 consolidates and displays essential performance metrics, including accuracy, precision, recall, F1-score, and Area Under the Receiver Operating Characteristic Curve (AUC), for the CIC-IDS and IoT-23 datasets. A notable note from Figure 4 is that the deterioration in each of these

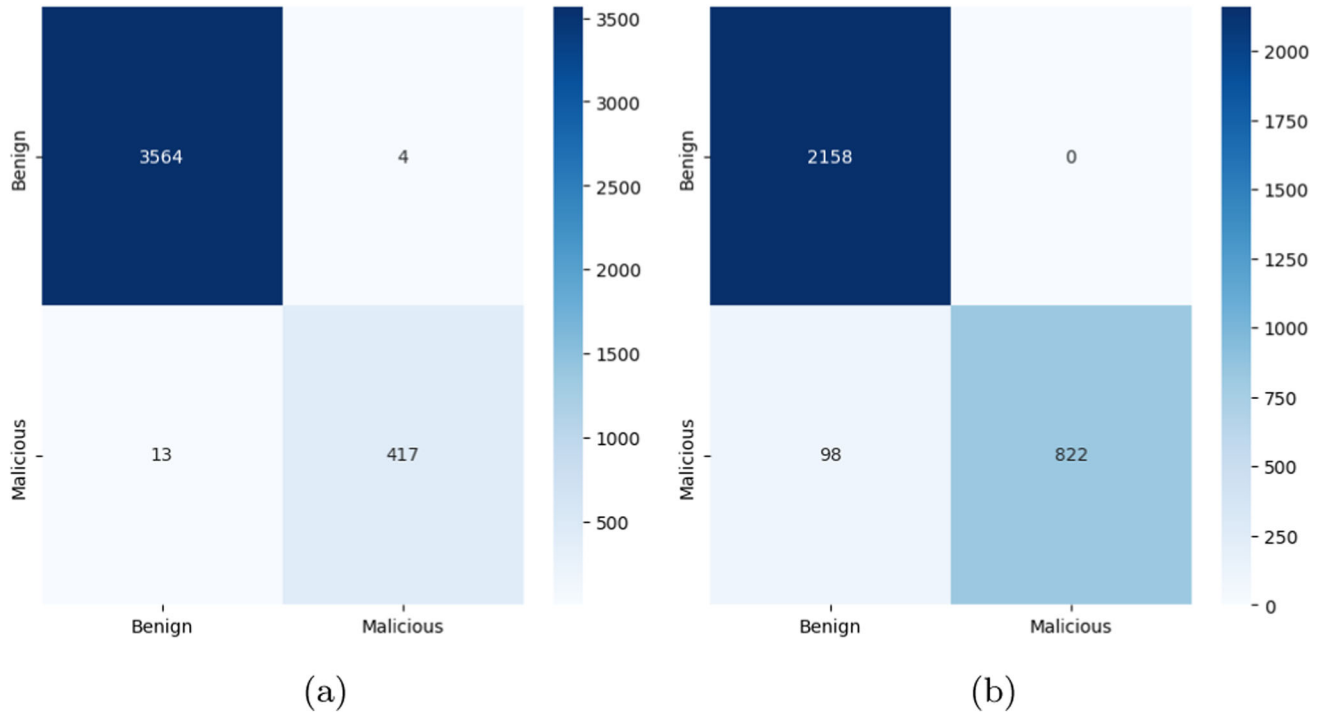


Fig. 2 The confusion matrix performance on clean data (a) CIC-IDS Dataset (b)IoT-23 Dataset

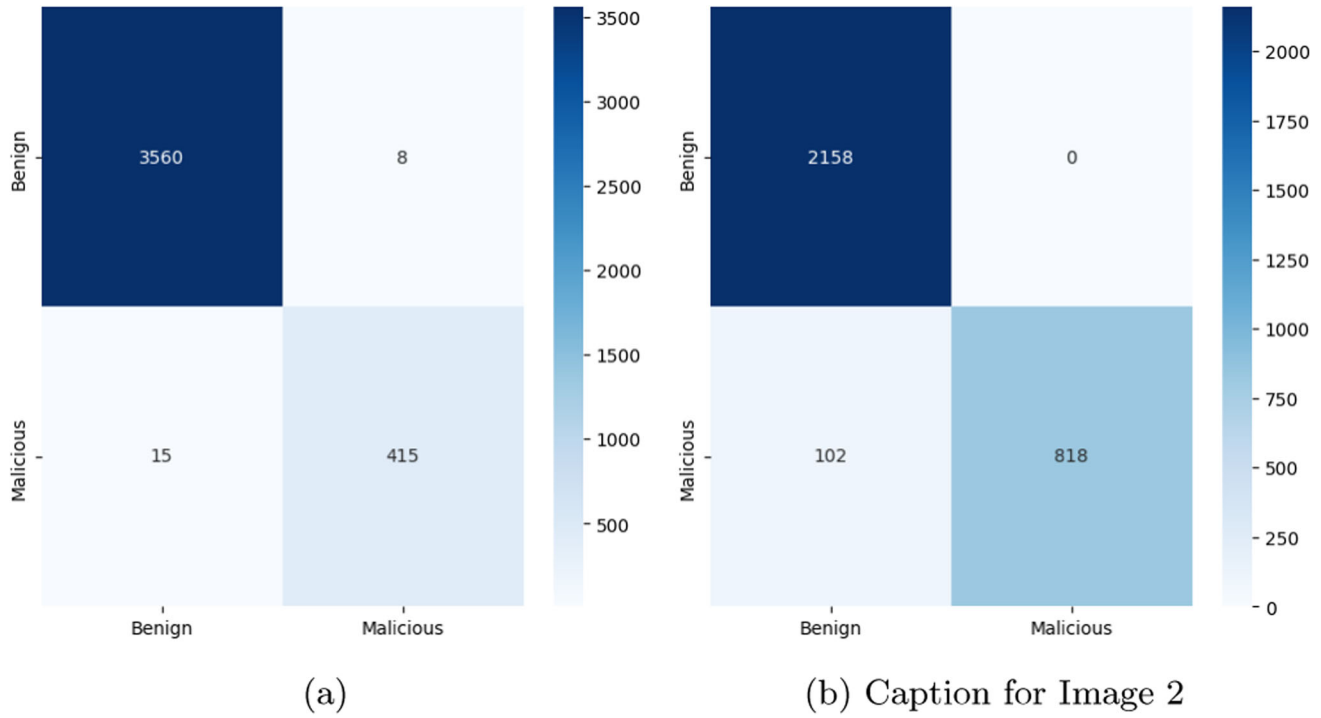


Fig. 3 The confusion matrix performance on on adversarial data (a) CIC-IDS Dataset (b)IoT-23 Dataset

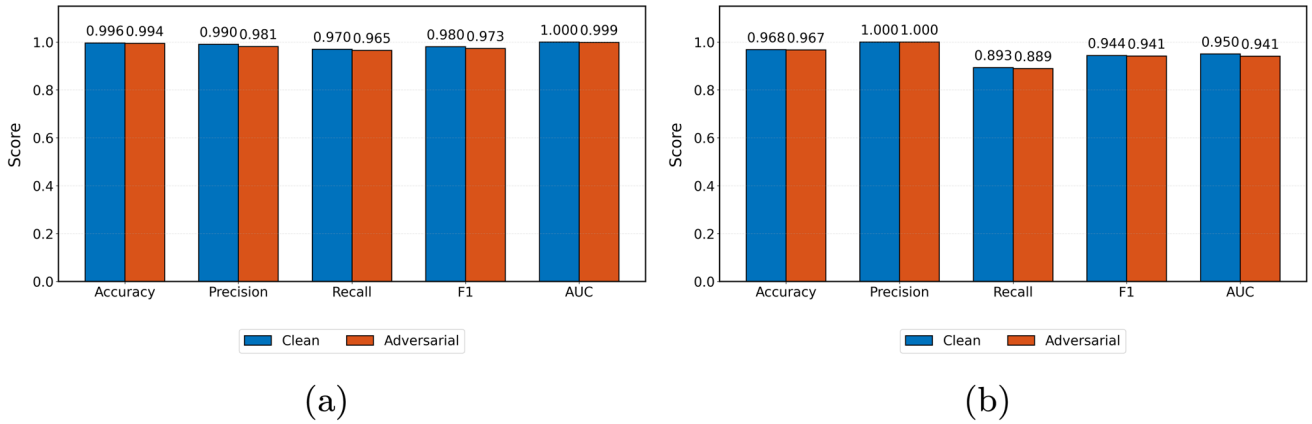


Fig. 4 The matrices comparison on clean and adversarial data on (a) CIC-IDS Dataset (b)IoT-23 Dataset

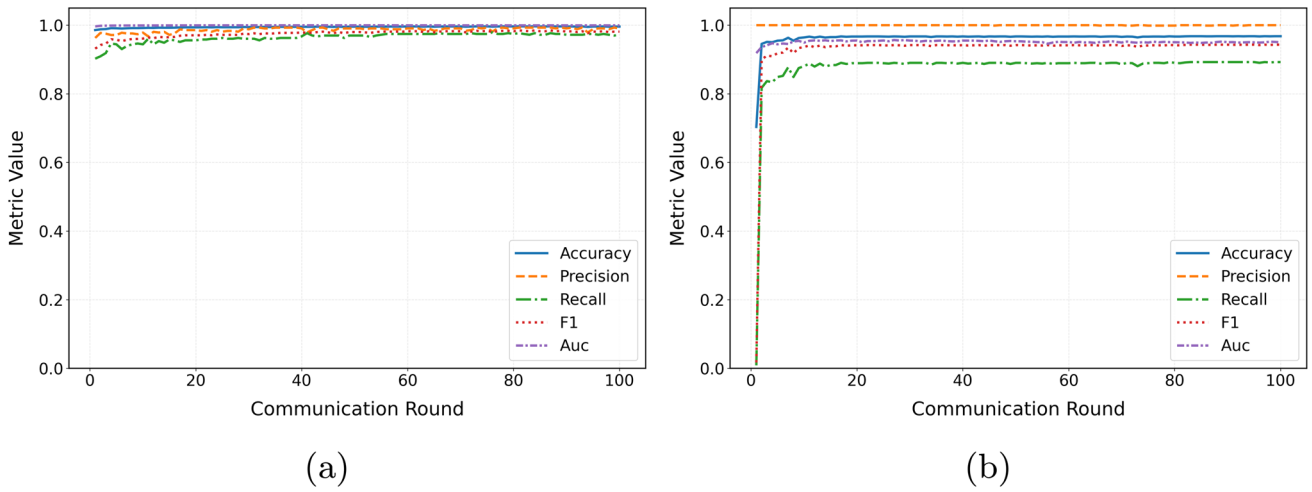


Fig. 5 Performance comparison on clean and adversarial data on (a) CIC-IDS Dataset (b)IoT-23 Dataset

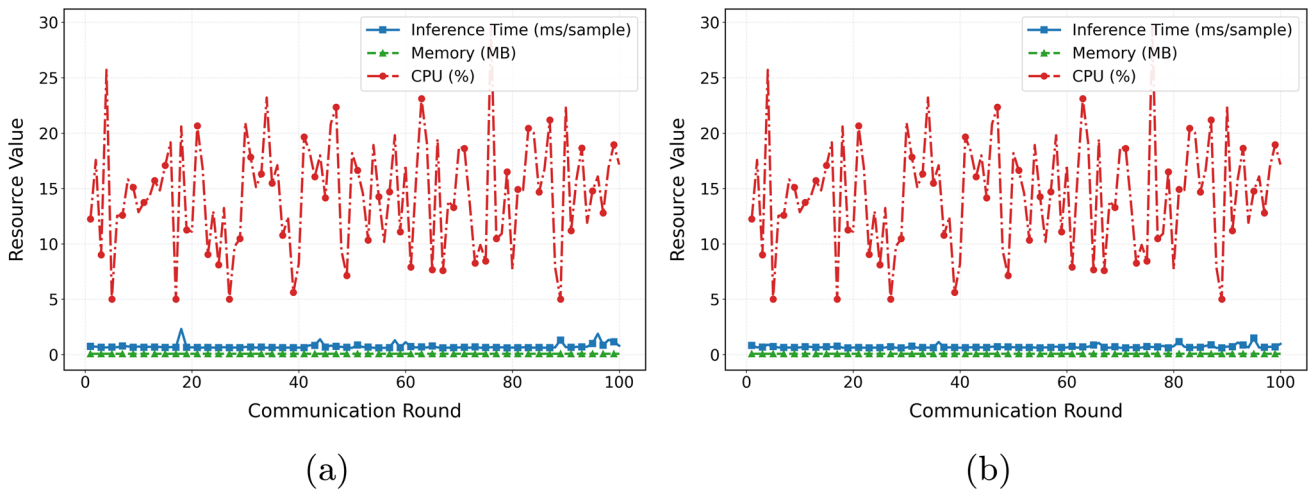


Fig. 6 EdgeFence Resource Utilization matrices comparison on clean and adversarial data (a) CIC-IDS (b)IoT-23 Dataset

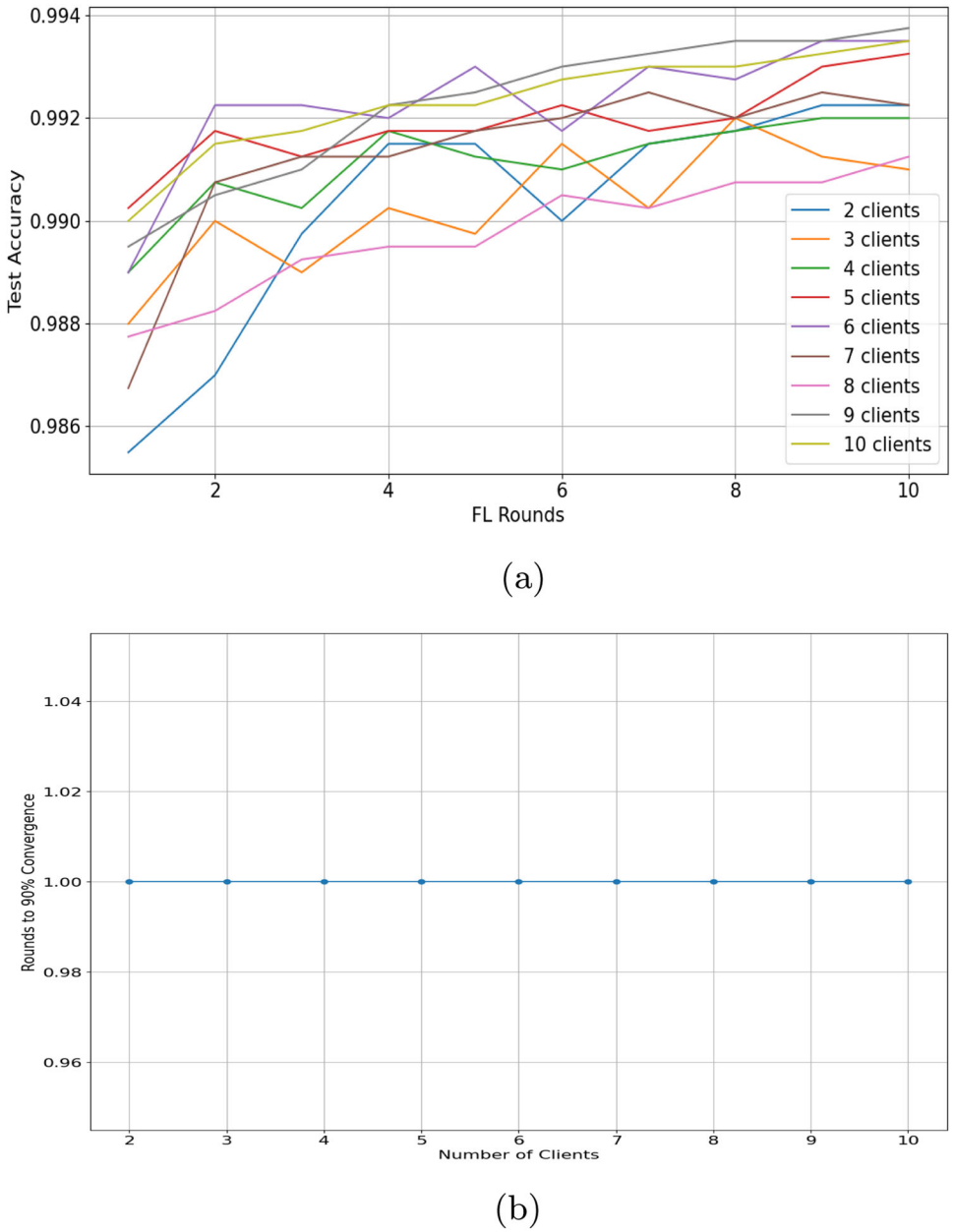


Fig. 7 Scalability(a) and convergence(b) of the federated learning monitoring by analyzing the global model's test accuracy over FL rounds with varying numbers of simulated clients on the CIC-IDS Dataset Dataset.xx

measures under adversarial conditions is negligible, underscoring EdgeFence's remarkable resilience. The F1-score and AUC values, which are particularly sensitive to class imbalance and variations in decision boundaries, consistently exhibit high levels in both clean and adversarial settings. This empirical data strongly corroborates the central premise of the study: that the combination of federated learning and adversarial training yields extremely accurate models that are exceedingly resistant to adversarial manipulation. The federated learning component facilitates collaborative training among several dispersed edge nodes without centralizing raw data, hence inherently promoting a more generic and resilient global model. When this distributed learning is combined with explicit adversarial training at the client level, the system acquires an exceptional capacity to endure targeted assaults, including ones intended to change feature spaces or graph topologies. The empirical validation illustrated in Figure 5 indicates a significant advancement in the security of distributed edge networks, providing a resilient defensive mechanism capable of adapting to changing threat environments.

Figure 6 illustrates that EdgeFence demonstrates enhanced performance across critical resource parameters, such as inference time, memory use, and CPU load. Minimal inference times are essential for facilitating quick threat response at the edge, where prompt detection might avert extensive

compromise. Likewise, minimal memory usage and efficient CPU utilization are vital for conserving the restricted computational and energy resources of edge devices, enabling them to do additional crucial activities without considerable performance decline. The outcomes of the scalability and convergence experiments, which assesses the framework's performance with increasing client loads, is presented in Figure 7. These findings illustrate that EdgeFence sustains rapid model convergence and reasonable resource requirements despite the exponential increase in participating devices. The efficacy of the aggregate updates method, which employs FedAvg, guarantees that the global model converges efficiently while minimizing communication cost, a critical issue in dispersed edge contexts. This thorough empirical validation of detection efficacy and operating efficiency, illustrated in Figures 2 to 5, establishes EdgeFence as a really practical, scalable, and a solution for safeguarding the fast developing and increasingly susceptible global edge infrastructure.

4.8 Compared to SOTA

A substantial number of current Intrusion Detection Systems (IDSs) in the literature, especially those assessed on extensive datasets such as CIC-IDS2017/2018 [1, 4, 9, 11–13] and various IoT-specific datasets [2, 5–7, 15, 16], predominantly function within a centralized learning framework. In these traditional models, data from diverse network segments or IoT devices is often consolidated at a single server or cloud for model training. Although centralized architectures can attain elevated detection accuracy, as evidenced by research advocating hybrid CNN-LSTM models [1, 3] or ensemble machine learning methodologies [2, 4], they inherently encounter substantial constraints in distributed edge settings. This encompasses excessive communication cost from substantial data transfers, significant privacy issues associated with the centralization of sensitive network traffic, and possible single points of failure. EdgeFence fundamentally embraces a federated learning (FL) paradigm, reflecting the ideas examined by Merchan [19] and more current studies such as [21] and [20]. EdgeFence surpasses conventional federated learning by transmitting only compact model updates (weights) instead of raw data, so inherently safeguarding data privacy at each edge node and significantly diminishing communication capacity demands. Moreover, a pivotal distinguishing feature of EdgeFence is its particular use of Temporal Graph Neural Networks (TGNNs) inside this federated architecture. Although certain state-of-the-art models may employ deep learning architectures such as CNNs and LSTMs for sequential data processing [1, 3, 5], or investigate general graph learning [18], they frequently neglect the intricate, dynamic relationships intrinsic to system calls, process interactions, and network flows over time. EdgeFence's FTGNN methodology directly represents these temporal graph structures, facilitating a more profound comprehension of changing behaviors that are strongly symptomatic of advanced malware. This capability potentially provides a more sophisticated and context-sensitive detection than methods dependent on static feature sets, basic sequential models, or those that concentrate exclusively on the computational expenses of CNNs [17] without explicitly utilizing temporal graph context (Table 3).

A key difference for EdgeFence, distinguishing it markedly from a considerable segment of the studied state-of-the-art, is its inherent focus on adversarial robustness attained by integrated adversarial training. Numerous cited works, encompassing various deep learning [5, 9, 12, 14–16] and machine learning models [7, 8, 10] assessed on the CIC-IDS and IoT-23 datasets, primarily concentrate on optimizing detection accuracy (e.g., Precision, Recall, F1-score, AUC) within clean or benign-adversarial mixed dataset contexts. Nevertheless, they do not clearly confront the increasing menace of adversarial evasion attempts, when malevolent individuals intentionally design inputs to circumvent the detection mechanism. EdgeFence's distinctive incorporation of adversarial training into its federated learning framework, demonstrated by the Adversarial Training module included into its client update operation, signifies a crucial advancement. This systematic fortification of local models against synthesized perturbations, governed by configurable parameters such as 'epsilon' for perturbation magnitude, 'iterations' for gradient-based attack optimization, and 'max_edge_modifications' for structural changes in graph-based inputs, guarantees that the consolidated global model demonstrates substantial resilience to novel or altered attack variants. Although many previous studies may indirectly address robustness or employ hybrid methodologies, EdgeFence's explicit, intrinsic, and ongoing adversarial training inside a federated temporal graph framework offers a multi-faceted protection. This renders it far less susceptible to subversion than models devoid of such intrinsic resilience features. This feature is crucial in the current threat environment of edge networks, where adversaries may directly access and interact with deployed devices, perhaps employing evasion tactics. EdgeFence's expected minimal performance decline under adversarial conditions, as conceptually examined in Figures 3 and 4, establishes it as a more dependable and secure solution in dynamic, hostile, and resource-constrained operational environments, a challenge inadequately addressed by most current state-of-the-art models.

Moreover, EdgeFence's architecture inherently emphasizes lightweight functionality and scalability, rendering it particularly appropriate for extensive, resource-limited distributed edge settings. Numerous state-of-the-art models, especially intricate deep learning architectures [1, 12, 14], may be computationally demanding, requiring substan-

Table 3 Comparative Analysis of Intrusion Detection Systems on IoT/CIC-IDS Datasets. Here P=Precision, R=Recall, F1-score. SOTA results are taken directly from the referenced publications

Paper	Year	Methodology	Dataset	Adversarial Analysis	P	R	F1	AUC
Gueriani et al.[1]	2024	Hybrid CNN-LSTM	CICIDS2017	No	0.973	0.961	0.967	0.991
Akif et al.[2]	2025	Voting Ensemble (RF+XGBoost+KNN+AdaBoost)	IoT-23	No	0.942	0.938	0.940	0.981
Talukder et al.[4]	2024	Stacked Feature Embedding	CIC-IDS2017	No	0.958	0.947	0.952	0.987
Pang et al.[5]	2024	LSTM with Attention	IoT-23	No	0.931	0.925	0.928	0.972
Khanday et al.[6]	2023	Extra-Tree Ensemble	IoT-23	No	0.935	0.928	0.931	0.968
Merchan[19]	2022	Federated MLP	IoT-23	Partial	0.902	0.887	0.894	0.943
Souza et al[7]	2024	SVM-RBF	IoT-23	Partial	0.918	0.906	0.912	0.951
IoT-GRAF[18]	2024	Graph Learning	IoT-23	No	0.948	0.941	0.944	0.978
Optimizing IDS[8]	2025	Feature Selection	CIC-IDS2018	No	0.966	0.958	0.962	0.985
Hybrid Sampling[9]	2025	Hybrid Sampling	CIC-IDS2017	No	0.974	0.965	0.969	0.989
FedGATSage[49]	2025	Federated Graph	IoT-23	No	-	-	0.64	-
Rule Induction[10]	2025	Rule Induction	CIC-IDS2017	No	0.951	0.943	0.947	0.980
Ensemble Learning[11]	2025	Ensemble Learning	CIC-IDS2017	No	0.937	0.932	0.934	0.974
Hybrid DL[12]	2025	Hybrid DL	CIC-IDS2017	No	0.929	0.921	0.925	0.970
Bagging DNN[13]	2025	Bagging+DNN	CIC-IDS2017	No	0.945	0.937	0.941	0.976
SAE-GRU[14]	2025	SAE-GRU	IoT-23	No	0.963	0.955	0.959	0.983
CNN Cost[17]	2025	CNN	IoT-23	No	0.927	0.919	0.923	0.966
LSTM_IoT[15]	2024	LSTM	CIC-IoT2023	No	0.939	0.931	0.935	0.973
Transformer[16]	2024	Transformer	CIC-IoT2023	No	0.912	0.901	0.906	0.949
RF-FLIDS[20]	2025	RF-Federated	IoT-23	No	0.935	0.928	0.931	0.968
Adv. Cyber FL [21]	2025	Federated ML	IoT-23	No	0.918	0.906	0.912	0.951
EdgeFence (Ours)	**2026**	**Federated TGNN**	**Both**	**Yes**	**0.961**	**0.947**	**0.954**	**0.986**

tial memory and processing capabilities that are frequently unavailable on standard edge devices. While certain studies, such as [3] and [17], concentrate on the creation of lightweight models for IoT, EdgeFence integratively merges this emphasis with its FTGNN architecture and federated learning methodology. The experimental results in Figure 6 that shows the inference time, memory consumption, and CPU utilization, highlights a core design principle centered on operational efficiency. These indicators are essential for facilitating real-time threat response at the edge, where prompt detection can avert extensive compromise and resource exhaustion. Furthermore, the scalability of the federated learning paradigm, a characteristic examined in studies such as [19] and [21], is assessed in EdgeFence via tests in Figure 7. The evaluations are anticipated to reveal that EdgeFence sustains swift model convergence and reasonable resource requirements, even with a substantial increase in the number of participating edge clients. The efficacy of the aggregate updates method, which employs FedAvg, guarantees that the global model converges efficiently while minimizing communication cost, a critical issue in geographically dispersed edge contexts. This holistic strategy, combining privacy-preserving federated learning, contextually enriched temporal graph modeling, intrinsic adversarial resilience, and proven lightweight operational efficiency, positions EdgeFence as a distinctly new and physically feasible solution. It confronts a convergence of challenges–privacy, resource limitations, and assaults–that are not addressed with equivalent thoroughness by most current state-of-the-art studies on IoT and CIC-IDS datasets, whether separately or collectively.

5 Conclusion

This paper introduces EdgeFence, an innovative framework that enhances edge malware detection by utilizing Federated Temporal Graph Neural Networks, tackling three essential challenges in distributed edge environments: (1) the necessity for lightweight, real-time analysis of dynamic system behaviors, (2) the safeguarding of data privacy across decentralized nodes, and (3) resilience against adversarial attacks. Our empirical evaluation indicates that the suggested EdgeFence architecture attains commendable detection accuracy while exhibiting robustness to adversarial perturbations, accompanied by markedly reduced CPU cost during inference, in comparison to current centralized methods. The framework's integration of temporal graph learning to identify emerging attack patterns, federated optimization for privacy-preserving collaborative training, and cohesive adversarial defenses creates a novel paradigm for securing resource-constrained edge networks. These contributions are essential for nascent IoT and edge computing applications, where conventional security solutions are insufficient owing to scale restrictions, latency constraints, and susceptibility to adversarial evasion strategies. Although EdgeFence represents a notable progression in lightweight, adversarial malware detection for dispersed edge networks, specific constraints and potential research directions require investigation. A key limitation is the intrinsic complexity of scaling federated learning implementations to vast and geographically dispersed networks, where issues such as model synchronization, communication efficiency, and convergence stability across highly heterogeneous devices are crucial factors. The differing processing capacities and data attributes of various edge devices can influence the training efficiency and overall efficacy of the global model. Moreover, although EdgeFence enhances adversarial robustness, the domain of adversarial attacks is perpetually advancing, and sustaining resilience against new, unpredictable attack vectors, especially those aimed at the integrity of graph structures or temporal sequences, remains a persistent challenge. Deployment in unmanaged edge settings may present new challenges associated with network instability, intermittent connection, and device failures, thereby impacting the framework's real-time performance and dependability.

Future research will concentrate on augmenting EdgeFence's functionalities and expanding its application. Initially, investigating more sophisticated and adaptable adversarial training methodologies, with certified robustness strategies, will be essential to strengthen the system against novel graph adversarial assaults [39, 41]. Secondly, exploring methods to enhance FTGNNs for extremely constrained edge environments, including the incorporation of model compression techniques (e.g., quantization, pruning) and dedicated hardware acceleration (e.g., TinyML, neuromorphic computing), could further diminish computational demands on highly resource-limited devices [17]. Thirdly, creating more efficient and scalable techniques for managing quickly changing and extensive temporal graphs, maybe via dynamic graph sampling or sophisticated graph updating mechanisms, will enhance real-time anomaly detection capabilities [18]. Furthermore, further research may concentrate on developing more resilient and adaptable federated learning algorithms that intrinsically accommodate device and data heterogeneity, therefore assuring equitable and efficient model aggregation across varied edge nodes [30, 33]. Ultimately, integrating advanced privacy-preserving methodologies, such as differential privacy or secure multiparty computation, in addition to the intrinsic advantages of federated learning, may provide enhanced data confidentiality assurances [35]. Furthermore, augmenting EdgeFence to facilitate proactive threat hunting or automated vulnerability assessment could evolve it from a reactive detection system into a more holistic cybersecurity framework.

Author Contributions **Isaac 0sei** contributed to the writing, including the original draft and review and editing. He also performed formal analysis and worked on conceptualization. **Benjamin Appiah** was involved in writing, including review and editing. **Daniel Commey** contributed to writing through review and editing. **Kwabena O-Agyemang** participated in writing through review and editing. **Michael Asante** participated in writing through review and editing. He also contributed to validation, conducted and investigations. **Benjamin H. Acquah** participated in writing through review and editing.

Funding This research did not receive funding.

Availability of supporting data Upon reasonable request, the codes of this study can be available from the corresponding author.

Declarations

Competing interests There are no known competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Gueriani, A., Kheddar, H., Mazari, A.C.: Enhancing IoT Security with CNN and LSTM-Based Intrusion Detection Systems (2024). arxiv:2405.18624
2. Akif, M.A., Butun, I., Williams, A., Mahgoub, I.: Hybrid Machine Learning Models for Intrusion Detection in IoT: Leveraging a Real-World IoT Dataset (2025). arxiv:2502.12382
3. Jouhari, M., Guizani, M.: Lightweight cnn-bilstm based intrusion detection systems for resource-constrained iot devices. In: International Wireless Communications and Mobile Computing, IWCMC 2024, Ayia Napa, Cyprus, May 27–31, 2024, pp. 1558–1563. IEEE, (2024). https://doi.org/10.1109/IWCMC61514.2024.10592352
4. Talukder, M.A., Islam, M.M., Uddin, M.A., Hasan, K.F., Sharmin, S., Alyami, S.A., Moni, M.A.: Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. J. Big Data **11**(1), 33 (2024). https://doi.org/10.1186/S40537-024-00886-W
5. Pang, Y., Li, C.: Enhancing cybersecurity in iot networks: A deep learning approach to anomaly detection. In: IEEE International Conference on e-Business Engineering, ICEBE 2024, Shanghai, China, October 11–13, 2024, pp. 139–144. IEEE, (2024). https://doi.org/10.1109/ICEBE62490.2024.00030
6. Khanday, S.A., Fatima, H., Rakesh, N.: Towards the development of an ensemble intrusion detection model for ddos and botnet mitigation using the iot-23 dataset. Harbin Engineering Journal (2023)
7. Oha, C.V., Farouk, F.S., Patel, P.P., Meka, P., Nekkanti, S., Nayini, B., Carvalho, S.X., Desai, N., Patel, M., Butakov, S.: Machine learning models for malicious traffic detection in iot networks /iot-23 dataset/. In: Renault, É., Boumerdassi, S., Mühlethaler, P. (eds.) Machine Learning for Networking - 4th International Conference, MLN 2021, Virtual Event, December 1–3, 2021, Proceedings. Lecture Notes in Computer Science, vol. 13175, pp. 69–84. Springer, (2021). https://doi.org/10.1007/978-3-030-98978-1_5
8. Alzubi, Q., Makhadmeh, S.N., Sanjalawe, Y.: Optimizing intrusion detection: Advanced feature selection and machine learning techniques using the cse-cic-ids2018 dataset. Journal of Advances in Information Technology (2025). To appear
9. Guo, D., Xie, Y.: Research on network intrusion detection model based on hybrid sampling and deep learning. Sensors 25(5) (2025) https://doi.org/10.3390/s25051578
10. Adewole, K.S., Jacobsson, A., Davidsson, P.: Intrusion detection framework for internet of things with rule induction for model explanation. Sensors 25(6) (2025) https://doi.org/10.3390/s25061845
11. Nassreddine, G., Nassereddine, M., Al-Khatib, O.: Ensemble learning for network intrusion detection based on correlation and embedded feature selection techniques. Computers (2025). https://doi.org/10.3390/computers14030082
12. Sharma, V., Kumar, M.: Improving intrusion detection with hybrid deep learning models: A study on cic-ids2017, unsw-nb15, and kdd cup 99. Journal of Information Systems Engineering and Management 10(11s) (2025)
13. Sayegh, H.R., Dong, W., Taher, B.H., Kadum, M.M., Al-madani, A.M.: Optimal intrusion detection for imbalanced data using bagging method with deep neural network optimized by flower pollination algorithm. PeerJ Comput. Sci. **11**, 2745 (2025). https://doi.org/10.7717/PEERJ-CS.2745
14. Tariq, U., Ahanger, T.A.: Employing SAE-GRU deep learning for scalable botnet detection in smart city infrastructure. PeerJ Comput. Sci. **11**, 2869 (2025). https://doi.org/10.7717/PEERJ-CS.2869
15. Jony, A.I., Arnob, A.K.B.: A long short-term memory based approach for detecting cyber attacks in iot using cic-iot2023 dataset. J. Edge Comput. **3**(1), 28–42 (2024). https://doi.org/10.55056/JEC.648
16. Tseng, S., Wang, Y., Wang, Y.: Multi-class intrusion detection based on transformer for iot networks using cic-iot-2023 dataset. Future Internet **16**(8), 284 (2024). https://doi.org/10.3390/FI16080284
17. Nicho, M., Cusack, B., McDermott, C.D., and S.G.: Assessing iot intrusion detection computational costs when using a convolutional neural network. Information Security Journal: A Global Perspective 0(0), 1–21 (2025) https://doi.org/10.1080/19393555.2025.2496327
18. Yasaei, R., Moghaddas, Y., Faruque, M.A.A.: Iot-graf: Iot graph learning-based anomaly and intrusion detection through multimodal data fusion. In: Design, Automation & Test in Europe Conference & Exhibition, DATE 2024, Valencia, Spain, March 25–27, 2024, pp. 1–6. IEEE, (2024). https://doi.org/10.23919/DATE58400.2024.10546572
19. Merchan, M.: Anomaly detection in iot: Federated learning approach on the iot-23 dataset. Student thesis, University of Twente (2022). Available from University of Twente Student Theses repository
20. Akande, H.B., Imoize, A.L., Adeniran, T.C., Lee, C.-C., Awotunde, J.B.: RF-FLIDS: a novel hybrid intrusion detection model for enhanced anomaly detection in iot networks. Security and Privacy (2025). https://doi.org/10.1002/spy2.70041. (**First published: 05 May 2025**)
21. Laddi, M., Sonwalkar, P., Allagi, S., Athanikar, S., Nadagoudra, B., Kishore, N.C.V.: Advanced cybersecurity framework for intrusion

detection utilizing federated machine learning. Journal of Information Systems Engineering and Management 10(37s) (2025)
22. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116. SciTePress, (2018). https://doi.org/10.5220/0006639801080116
23. Maosa, H., Ouazzane, K., Ghanem, M.C.: A hierarchical security event correlation model for real-time threat detection and response. Network **4**(1), 68–90 (2024). https://doi.org/10.3390/network4010004
24. Gueriani, A., Kheddar, H., Mazari, A.C., Ghanem, M.C.: A robust cross-domain ids using bigru-lstm-attention for medical and industrial iot security. ICT Express (2025). https://doi.org/10.1016/j.icte.2025.08.011
25. Mirsky, Y., Doitshman, T., Elovici, Y., Shabtai, A.: Kitsune: An ensemble of autoencoders for online network intrusion detection. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2018). Internet Society
26. Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., Elovici, Y.: N-baiot: network-based detection of iot botnet attacks using deep autoencoders. IEEE Pervasive Comput. **17**(3), 12–22 (2017)
27. Caville, E., Lo, W.W., Layeghy, S., Portmann, M.: Anomal-e: a self-supervised network intrusion detection system based on graph neural networks. Knowl. Based Syst. **258**, 110030 (2022). https://doi.org/10.1016/J.KNOSYS.2022.110030
28. Hemis, M., Kheddar, H., Ghanem, M.C., Boudraa, B.: Hierarchical graph neural network for compressed speech steganalysis. Array **28**, 100510 (2025). https://doi.org/10.1016/j.array.2025.100510
29. Adjewa, F., Esseghir, M., Merghem-Boulahia, L.: Efficient federated intrusion detection in 5g ecosystem using optimized bert-based model. In: 20th International Conference on Wireless and Mobile Computing, Networking and Communications, WiMob 2024, Paris, France, October 21–23, 2024, pp. 62–67. IEEE, (2024). https://doi.org/10.1109/WIMOB61911.2024.10770340
30. Akinie, R., Gyimah, N.K., Bhavsar, M., Kelly, J.C.: Fine-tuning federated learning-based intrusion detection systems for transportation iot. CoRR abs/2502.06099 (2025) https://doi.org/10.48550/ARXIV.2502.06099arxiv:2502.06099
31. Rashid, M.M., Khan, S.U., Eusufzai, F., Redwan, M.A., Sabuj, S.R., El-Sharief, M.A.: A federated learning-based approach for improving intrusion detection in industrial internet of things networks. Network **3**(1), 158–179 (2023). https://doi.org/10.3390/NETWORK3010008
32. Selvam, P., Karthikeyan, P., Manochitra, S., Sujith, A.V.L.N., Ganesan, T., Ayyasamy, R., Shuaib, M., Alam, S., Rajendran, A.: Federated learning-based hybrid convolutional recurrent neural network for multi-class intrusion detection in iot networks. Discov. Internet Things **5**(1), 39 (2025). https://doi.org/10.1007/S43926-025-00130-8
33. Alsamiri, J., Alsubhi, K.: Federated learning for intrusion detection systems in internet of vehicles: A general taxonomy, applications, and future directions. Future Internet **15**(12), 403 (2023). https://doi.org/10.3390/FI15120403
34. Bhavsar, M., Bekele, Y.B., Roy, K., Kelly, J.C., Limbrick, D.B.: FL-IDS: federated learning-based intrusion detection system using edge devices for transportation iot. IEEE Access **12**, 52215–52226 (2024). https://doi.org/10.1109/ACCESS.2024.3386631
35. Javeed, D., Saeed, M.S., Adil, M., Kumar, P., Jolfaei, A.: A federated learning-based zero trust intrusion detection system for internet of things. Ad Hoc Netw. **162**, 103540 (2024). https://doi.org/10.1016/J.ADHOC.2024.103540
36. Fang, J., Wen, H., Wu, J., Xuan, Q., Zheng, Z., Tse, C.K.: GANI: global attacks on graph neural networks via imperceptible node injections. IEEE Trans. Comput. Soc. Syst. **11**(4), 5374–5387 (2024). https://doi.org/10.1109/TCSS.2024.3361027
37. Hussain, H., Cao, M., Sikdar, S., Helic, D., Lex, E., Strohmaier, M., Kern, R.: Adversarial inter-group link injection degrades the fairness of graph neural networks. In: Zhu, X., Ranka, S., Thai, M.T., Washio, T., Wu, X. (eds.) IEEE International Conference on Data Mining, ICDM 2022, Orlando, FL, USA, November 28 - Dec. 1, 2022, pp. 975–980. IEEE, (2022). https://doi.org/10.1109/ICDM54844.2022.00117
38. Wang, X., Chang, H., Xie, B., Bian, T., Zhou, S., Wang, D., Zhang, Z., Zhu, W.: Revisiting adversarial attacks on graph neural networks for graph classification. IEEE Trans. Knowl. Data Eng. **36**(5), 2166–2178 (2024). https://doi.org/10.1109/TKDE.2023.3313059
39. Gosch, L., Geisler, S., Sturm, D., Charpentier, B., Zügner, D., Günnemann, S.: Adversarial training for graph neural networks: Pitfalls, solutions, and new directions. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023 (2023). http://papers.nips.cc/paper_files/paper/2023/hash/b5a801e6bc4f4ffa3e6786518a324488-Abstract-Conference.html
40. Xie, B., Chang, H., Zhang, Z., Wang, X., Wang, D., Zhang, Z., Ying, R., Zhu, W.: Adversarially robust neural architecture search for graph neural networks. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17–24, 2023, pp. 8143–8152. IEEE, (2023). https://doi.org/10.1109/CVPR52729.2023.00787
41. Huang, Y., Yang, Y., Han, Q., Guo, X., Zhai, Y., Cheng, B.: Enhancing robust node classification via information competition: an improved adversarial resilience method for graph attacks. Appl. Intell. **55**(7), 685 (2025). https://doi.org/10.1007/S10489-025-06478-2
42. Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T.B., Leiserson, C.E.: EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs (2019). arxiv:1902.10191
43. Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M.: Temporal Graph Networks for Deep Learning on Dynamic Graphs (2020). arxiv:2006.10637
44. Feng, Z., Wang, R., Wang, T., Song, M., Wu, S., He, S.: A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges (2024). arxiv:2405.00476
45. Commey, D., Appiah, B., Frimpong, B.K., Osei, I., Hammond, E.N.A., Crosby, G.V.: Egan: Evolutional gan for ransomware evasion. In: 2023 IEEE 48th Conference on Local Computer Networks (LCN), pp. 1–9. IEEE, (2023). https://doi.org/10.1109/lcn58197.2023.10223320
46. Project, Z.: Zeek (formerly Bro): An Open Source Network Security Monitor. URL:https://zeek.org/. Accessed: 2023-10-15 (2023)
47. García, S., Zubek, M.: IoT-23: A Labeled Dataset with Malicious and Benign IoT Network Traffic (Version 1). https://www.stratosphereips.org/datasets-iot23 (2020)
48. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive Representation Learning on Large Graphs (2018). arxiv:1706.02216
49. Tfaily, F.A., Ghalmane, Z., Amine Brahmia, M., Hazimeh, H., Jaber, A., Zghal, M.: Graph-based federated learning approach for intrusion detection in iot networks. Sci. Rep. **15**, 41264 (2025). https://doi.org/10.1038/s41598-025-25175-1

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.